

5



Phương pháp lấy mẫu lại

Các phương pháp lấy mẫu lại là một công cụ không thể thiếu trong thống kê hiện đại. Chúng bao gồm việc liên tục lấy mẫu từ một tập dữ liệu huấn luyện và điều chỉnh lại mô hình cần quan tâm trên mỗi mẫu để thu được thông tin bổ sung về mô hình đã được điều chỉnh. Ví dụ, để ước tính sự biến thiên của một mô hình hồi quy tuyến tính, chúng ta có thể liên tục lấy các mẫu khác nhau từ dữ liệu huấn luyện, điều chỉnh hồi quy tuyến tính cho mỗi mẫu mới, và sau đó kiểm tra mức độ khác biệt giữa các kết quả điều chỉnh. Cách tiếp cận như vậy có thể cho phép chúng ta thu được thông tin mà không thể có được nếu chỉ điều chỉnh mô hình một lần bằng cách sử dụng mẫu huấn luyện ban đầu.

Các phương pháp lấy mẫu lại có thể tốn kém về mặt tính toán, bởi vì chúng liên quan đến việc áp dụng cùng một phương pháp thống kê nhiều lần bằng cách sử dụng các tập con khác nhau của dữ liệu huấn luyện. Tuy nhiên, nhờ những tiến bộ gần đây về sức mạnh tính toán, yêu cầu tính toán của các phương pháp lấy mẫu lại nhìn chung không còn quá lớn. Trong chương này, chúng ta sẽ thảo luận về hai trong số các phương pháp lấy mẫu lại được sử dụng phổ biến nhất, đó là kiểm định chéo và bootstrap. Cả hai phương pháp đều là những công cụ quan trọng trong ứng dụng thực tiễn của nhiều quy trình học thống kê. Ví dụ, kiểm định chéo có thể được sử dụng để ước tính sai số kiểm định liên quan đến một phương pháp học thống kê nhất định nhằm đánh giá hiệu suất của nó, hoặc để chọn mức độ linh hoạt phù hợp. Quá trình đánh giá hiệu suất của một mô hình được gọi là đánh giá mô hình, trong khi quá trình chọn mức độ linh hoạt phù hợp cho một mô hình được gọi là lựa chọn mô hình. Bootstrap được sử dụng trong nhiều ngữ cảnh, phổ biến nhất là để cung cấp thước đo độ chính xác của ước lượng tham số hoặc của một phương pháp học thống kê nhất định.

người mẫu
đánh giá
người mẫu
lựa chọn

5.1 Kiểm định chéo

Trong Chương 2, chúng ta sẽ thảo luận về sự khác biệt giữa tỷ lệ lỗi kiểm thử và tỷ lệ lỗi huấn luyện. Lỗi kiểm thử là lỗi trung bình phát sinh khi sử dụng phương pháp học thống kê để dự đoán phản hồi trên một quan sát mới – tức là một phép đo chưa được sử dụng trong quá trình huấn luyện phương pháp. Với một tập dữ liệu nhất định, việc sử dụng một phương pháp học thống kê cụ thể được coi là hợp lý nếu nó dẫn đến tỷ lệ lỗi kiểm thử thấp. Lỗi kiểm thử có thể dễ dàng được tính toán nếu có sẵn một tập dữ liệu kiểm thử được chỉ định. Tuy nhiên, điều này thường không xảy ra.

Ngược lại, sai số huấn luyện có thể dễ dàng tính toán bằng cách áp dụng phương pháp học thống kê vào các quan sát được sử dụng trong quá trình huấn luyện. Nhưng như chúng ta đã thấy trong Chương 2, tỷ lệ sai số huấn luyện thường khá khác biệt so với tỷ lệ sai số kiểm thử, và đặc biệt là tỷ lệ sai số huấn luyện có thể đánh giá thấp hơn đáng kể tỷ lệ sai số kiểm thử.

Trong trường hợp không có bộ dữ liệu kiểm thử lớn được chỉ định để trực tiếp ước tính tỷ lệ lỗi kiểm thử, một số kỹ thuật có thể được sử dụng để ước tính đại lượng này bằng cách sử dụng dữ liệu huấn luyện có sẵn. Một số phương pháp thực hiện điều chỉnh toán học đối với tỷ lệ lỗi huấn luyện để ước tính tỷ lệ lỗi kiểm thử. Các phương pháp như vậy được thảo luận trong Chương 6.

Trong phần này, chúng ta xem xét một nhóm các phương pháp ước tính tỷ lệ lỗi kiểm thử bằng cách giữ lại một tập hợp con các quan sát huấn luyện từ quá trình huấn luyện, và sau đó áp dụng phương pháp học thống kê cho các quan sát được giữ lại đó.

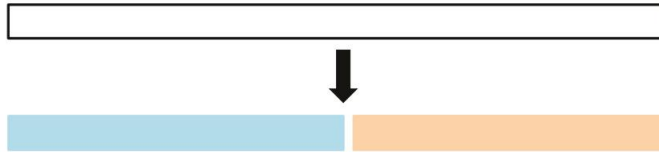
Trong các phần 5.1.1-5.1.4, để đơn giản, chúng ta giả định rằng chúng ta quan tâm đến việc thực hiện hồi quy với biến phản hồi định lượng. Trong phần 5.1.5, chúng ta xem xét trường hợp phân loại với biến phản hồi định tính. Như chúng ta sẽ thấy, các khái niệm chính vẫn giữ nguyên bất kể biến phản hồi là định lượng hay định tính.

5.1.1 Phương pháp tập dữ liệu kiểm định

Giả sử chúng ta muốn ước tính sai số kiểm định liên quan đến việc áp dụng một phương pháp học thống kê cụ thể lên một tập hợp các quan sát. Phương pháp tập dữ liệu kiểm định, được minh họa trong Hình 5.1, là một chiến lược rất đơn giản cho nhiệm vụ này. Nó bao gồm việc chia ngẫu nhiên tập hợp các quan sát có sẵn thành hai phần, một tập huấn luyện và một tập kiểm định hoặc tập giữ lại. Mô hình được áp dụng trên tập huấn luyện, và mô hình đã được áp dụng được sử dụng để dự đoán các phản hồi cho các quan sát trong tập kiểm định. Tỷ lệ sai số của tập kiểm định thu được—thường được đánh giá bằng MSE trong trường hợp phản hồi định lượng—cung cấp ước tính về tỷ lệ sai số kiểm định.

phương pháp
tập hợp xác thực
xác thực
bộ
bộ giữ

Chúng tôi minh họa phương pháp tập dữ liệu kiểm định trên tập dữ liệu *Auto*. Hãy nhớ lại từ Chương 3 rằng đường như có mối quan hệ phi tuyến tính giữa *mpg* và *mã lực*, và mô hình dự đoán *mpg* bằng cách sử dụng *mã lực* và *mã lực²* cho kết quả tốt hơn so với mô hình chỉ sử dụng một hệ số tuyến tính. Việc tự hỏi liệu một mô hình bậc ba hoặc bậc cao hơn có thể cho kết quả tốt hơn nữa hay không là điều dễ hiểu. Chúng ta sẽ trả lời câu hỏi này trong Chương 3 bằng cách xem xét các giá trị p liên quan đến số hạng bậc ba và các số hạng đa thức bậc cao hơn trong hồi quy tuyến tính. Nhưng chúng ta cũng có thể trả lời câu hỏi này bằng phương pháp kiểm định. Chúng ta chia ngẫu nhiên 392 quan sát thành hai nhóm.



HÌNH 5.1. Sơ đồ minh họa phương pháp tập dữ liệu kiểm định. Một tập hợp gồm n quan sát được chia ngẫu nhiên thành tập huấn luyện (màu xanh lam, bao gồm các quan sát 7, 22 và 13, cùng một số quan sát khác) và tập kiểm định (màu be, bao gồm quan sát 91, cùng một số quan sát khác). Phương pháp học thống kê được huấn luyện trên tập huấn luyện, và hiệu suất của nó được đánh giá trên tập kiểm định.

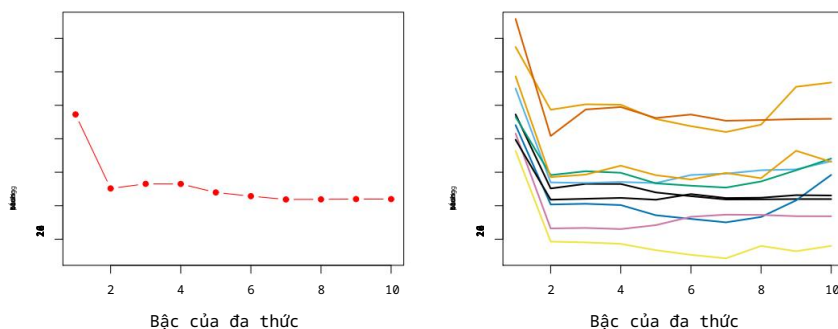
Tập dữ liệu được chia thành hai tập: tập huấn luyện chứa 196 điểm dữ liệu và tập kiểm định chứa 196 quan sát còn lại. Tỷ lệ lỗi của tập kiểm định thu được từ việc áp dụng các mô hình hồi quy khác nhau trên mẫu huấn luyện và đánh giá hiệu suất của chúng trên mẫu kiểm định, sử dụng MSE làm thước đo lỗi của tập kiểm định, được thể hiện ở bảng bên trái của Hình 5.2. MSE của tập kiểm định đối với mô hình hồi quy bậc hai nhỏ hơn đáng kể so với mô hình hồi quy tuyến tính. Tuy nhiên, MSE của tập kiểm định đối với mô hình hồi quy bậc ba lại lớn hơn một chút so với mô hình hồi quy bậc hai. Điều này ngụ ý rằng việc thêm số hạng bậc ba vào hồi quy không dẫn đến dự đoán tốt hơn so với việc chỉ sử dụng số hạng bậc hai.

Hãy nhớ rằng để tạo ra bảng bên trái của Hình 5.2, chúng ta đã chia ngẫu nhiên tập dữ liệu thành hai phần: tập huấn luyện và tập kiểm định. Nếu chúng ta lặp lại quá trình chia ngẫu nhiên tập mẫu thành hai phần, chúng ta sẽ nhận được một ước tính hơi khác cho MSE kiểm định. Để minh họa, bảng bên phải của Hình 5.2 hiển thị mười đường cong MSE kiểm định khác nhau từ tập dữ liệu *Auto*, được tạo ra bằng cách sử dụng mười cách chia ngẫu nhiên khác nhau các quan sát thành tập huấn luyện và tập kiểm định. Cả mười đường cong đều cho thấy mô hình có số hạng bậc hai có MSE kiểm định nhỏ hơn đáng kể so với mô hình chỉ có số hạng tuyến tính. Hơn nữa, cả mười đường cong đều cho thấy không có nhiều lợi ích khi bao gồm các số hạng đa thức bậc ba hoặc bậc cao hơn trong mô hình. Những điều đáng chú ý là mỗi trong mười đường cong đều cho ra một ước tính MSE kiểm định khác nhau cho mỗi trong mười mô hình hồi quy được xem xét. Và không có sự đồng thuận giữa các đường cong về mô hình nào cho ra MSE kiểm định nhỏ nhất.

Dựa trên sự khác biệt giữa các đường cong này, điều duy nhất chúng ta có thể kết luận một cách chắc chắn là phương pháp khớp tuyến tính không phù hợp với dữ liệu này.

Phương pháp sử dụng tập dữ liệu kiểm định có khái niệm đơn giản và dễ thực hiện. Tuy nhiên, nó có hai nhược điểm tiềm tàng:

1. Như thể hiện ở bảng bên phải của Hình 5.2, ước tính xác thực về tỷ lệ lỗi kiểm thử có thể rất khác nhau, tùy thuộc vào việc chính xác những quan sát nào được đưa vào tập huấn luyện và những quan sát nào được đưa vào tập xác thực.
2. Trong phương pháp kiểm định, chỉ một tập hợp con các quan sát—những quan sát được bao gồm trong tập huấn luyện chứ không phải trong tập kiểm định—được sử dụng để huấn luyện mô hình. Vì các phương pháp thống kê thường hoạt động kém hiệu quả hơn khi được huấn luyện trên ít quan sát hơn, điều này cho thấy rằng



HÌNH 5.2. Phương pháp tập dữ liệu xác thực đã được sử dụng trên tập dữ liệu *Auto* trong để ước tính sai số kiểm định phát sinh từ việc dự đoán *mpg* bằng đa thức chức năng của *mã lực*. Bên trái: Ước tính lỗi xác thực cho một lần chia thành Bộ dữ liệu huấn luyện và xác thực. Bên phải: Phương pháp xác thực được lặp lại mười lần. nhiều lần, mỗi lần sử dụng một cách chia ngẫu nhiên khác nhau các quan sát thành tập dữ liệu huấn luyện. một tập dữ liệu và một tập dữ liệu kiểm định. Điều này minh họa sự biến động trong MSE kiểm định ước tính. Đó là kết quả từ cách tiếp cận này.

Tỷ lệ lỗi trên tập dữ liệu xác thực có thể có xu hướng đánh giá quá cao tỷ lệ lỗi trên tập dữ liệu kiểm thử. để mô hình phù hợp với toàn bộ tập dữ liệu.

Trong các phần tiếp theo, chúng ta sẽ trình bày về phương pháp kiểm định chéo, một sự cải tiến của Phương pháp sử dụng tập dữ liệu kiểm định giải quyết được hai vấn đề này.

5.1.2 Phương pháp kiểm định chéo loại trừ từng mẫu (Leave-One-Out Cross-Validation)

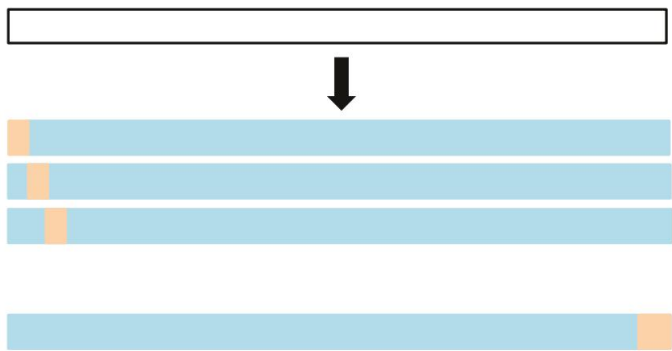
Phương pháp kiểm định chéo loại trừ một mẫu (LOOCV) có liên quan mật thiết đến phương pháp tập dữ liệu kiểm định trong Mục 5.1.1, nhưng nó cố gắng giải quyết những hạn chế của phương pháp đó. nhược điểm.

Tương tự như phương pháp tập dữ liệu xác thực, LOOCV cũng bao gồm việc chia tập dữ liệu. các quan sát được chia thành hai phần. Tuy nhiên, thay vì tạo ra hai tập con của Với kích thước tương đương, một quan sát duy nhất (x_1, y_1) được sử dụng để xác thực. tập hợp, và các quan sát còn lại $\{(x_2, y_2), \dots, (x_n, y_n)\}$ tạo thành tập huấn luyện. Phương pháp học thống kê được áp dụng trên tập huấn luyện $n - 1$ các quan sát, và một dự đoán $\hat{y}_1$ được đưa ra cho quan sát bị loại trừ. sử dụng giá trị x_1 của nó. Vì (x_1, y_1) không được sử dụng trong quá trình khớp dữ liệu, nên $MSE_1 = (y_1 - \hat{y}_1)^2$ cung cấp một ước tính gần như không thiên lệch cho sai số kiểm định. Tuy nhiên, mặc dù MSE_1 không thiên lệch đối với lỗi kiểm định, nó vẫn là một ước lượng kém chính xác. vì nó có tính biến động cao, do dựa trên một quan sát duy nhất.

(x_1, y_1) .

Chúng ta có thể lặp lại quy trình bằng cách chọn (x_2, y_2) để kiểm định. dữ liệu, huấn luyện quy trình học thống kê trên $n - 1$ quan sát $\{(x_1, y_1), (x_3, y_3), \dots, (x_n, y_n)\}$, và tính $MSE_2 = (y_2 - \hat{y}_2)^2$. Lặp lại phương pháp này n lần sẽ tạo ra n lỗi bình phương, $MSE_1, \dots, MSE_n$. Ước tính LOOCV cho MSE kiểm định là trung bình của n lỗi kiểm định này. ước tính:

$$CV(n) = \frac{1}{N} \sum_{i=1}^N MSE_i. \quad (5.1)$$



HÌNH 5.3. Sơ đồ minh họa phương pháp LOOCV. Một tập hợp gồm n điểm dữ liệu được chia lặp đi lặp lại thành một tập huấn luyện (màu xanh lam) chứa tất cả các quan sát trừ một quan sát, và một tập kiểm định chỉ chứa quan sát đó (màu be). Sai số kiểm định sau đó được ước tính bằng cách lấy trung bình n giá trị MSE thu được. Tập huấn luyện đầu tiên chứa tất cả các quan sát trừ quan sát 1, tập huấn luyện thứ hai chứa tất cả các quan sát trừ quan sát 2, và cứ thế tiếp tục.

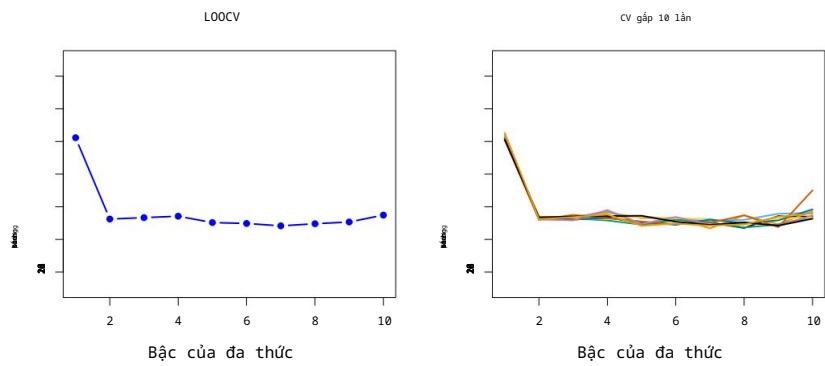
Sơ đồ minh họa phương pháp LOOCV được thể hiện trong Hình 5.3.

LOOCV có một vài ưu điểm vượt trội so với phương pháp sử dụng tập dữ liệu kiểm định. Thứ nhất, nó có độ sai lệch thấp hơn nhiều. Trong LOOCV, chúng ta liên tục huấn luyện phương pháp học thống kê bằng cách sử dụng các tập huấn luyện chứa $n - 1$ quan sát, gần bằng số lượng quan sát trong toàn bộ tập dữ liệu. Điều này trái ngược với phương pháp sử dụng tập dữ liệu kiểm định, trong đó tập huấn luyện thường chỉ bằng khoảng một nửa kích thước của tập dữ liệu gốc. Do đó, phương pháp LOOCV có xu hướng không đánh giá quá cao tỷ lệ lỗi kiểm thử nhiều như phương pháp sử dụng tập dữ liệu kiểm định. Thứ hai, trái ngược với phương pháp kiểm định sẽ cho ra các kết quả khác nhau khi áp dụng lặp đi lặp lại do tính ngẫu nhiên trong việc phân chia tập huấn luyện/kiểm định, việc thực hiện LOOCV nhiều lần sẽ luôn cho ra cùng một kết quả: không có tính ngẫu nhiên trong việc phân chia tập huấn luyện/kiểm định.

Chúng tôi đã sử dụng phương pháp LOOCV trên tập dữ liệu **Auto** để ước tính MSE của tập dữ liệu thử nghiệm, kết quả thu được từ việc áp dụng mô hình hồi quy tuyến tính để dự đoán **mpg** bằng cách sử dụng các hàm đa thức của **mã lực**. Kết quả được hiển thị ở bảng bên trái của Hình 5.4.

Phương pháp LOOCV có tiềm năng tốn kém để triển khai, vì mô hình phải được huấn luyện n lần. Điều này có thể rất tốn thời gian nếu n lớn và nếu mỗi mô hình riêng lẻ được huấn luyện chậm. Với phương pháp hồi quy tuyến tính hoặc đa thức bình phương tối thiểu, một phương pháp rút gọn tuyệt vời giúp giảm chi phí của LOOCV xuống bằng với chi phí huấn luyện một mô hình duy nhất! Công thức sau đây đúng:

$$CV(n) = \frac{1}{N} \sum_{i=1}^N \frac{y_i - \hat{y}_{i-1}^2}{\text{xin chào}}, \tag{5.2}$$



HÌNH 5.4. Phương pháp kiểm định chéo đã được sử dụng trên tập dữ liệu *Auto* để ước tính sai số kiểm định phát sinh từ việc dự đoán *mpg* bằng cách sử dụng các hàm đa thức mã *lực*. Bên trái: Đường cong lỗi LOOCV. Bên phải: CV gấp 10 lần được chạy chín lần các thời điểm khác nhau, mỗi thời điểm lại chia dữ liệu ngẫu nhiên thành mười phần khác nhau. Hình vẽ thể hiện chín đường cong lỗi CV hơi khác nhau.

trong đó y_i là giá trị được điều chỉnh thứ i từ phép điều chỉnh bình phương tối thiểu ban đầu, và h_i là đòn bẩy được xác định trong (3.37) trên trang 105. ¹ Đây giống như MSE thông thường, ngoại trừ phần dư thứ i được chia cho $1 - h_i$. Đòn bẩy nằm trong khoảng $1/n$ và 1, và phản ánh mức độ ảnh hưởng của một quan sát đến sự phù hợp của chính nó. Do đó, phần dư đối với các điểm có đòn bẩy cao bị thổi phồng trong công thức này bởi đây chính xác là lượng vừa đủ để sự cân bằng này được duy trì.

LOOCV là một phương pháp rất tổng quát và có thể được sử dụng với bất kỳ loại nào. Mô hình dự đoán. Ví dụ, chúng ta có thể sử dụng nó với hồi quy logistic. hoặc phân tích phân biệt tuyến tính, hoặc bất kỳ phương pháp nào được thảo luận ở phần sau. chương. Công thức ma thuật (5.2) không đúng trong trường hợp đó. Mô hình cần được hiệu chỉnh lại n lần.

5.1.3 Kiểm định chéo k-fold

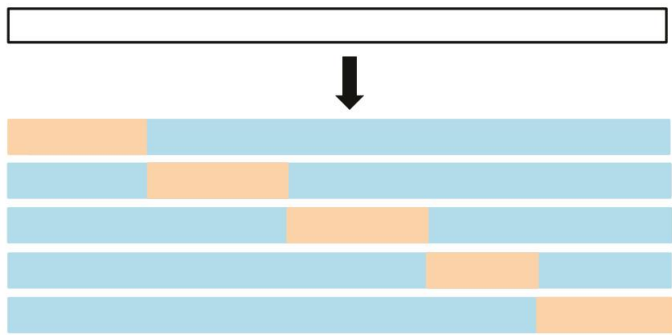
Một phương pháp thay thế cho LOOCV là k-fold CV. Phương pháp này bao gồm việc chia ngẫu nhiên tập hợp các quan sát thành k nhóm, hay các fold, có kích thước xấp xỉ k . kích thước bằng nhau. Tập dữ liệu đầu tiên được coi là tập dữ liệu xác thực, và phương pháp được khớp trên $k - 1$ fold còn lại. Sai số bình phương trung bình, MSE_1 , là sau đó được tính toán dựa trên các quan sát trong tập dữ liệu được giữ lại. Quy trình này là Lặp lại k lần; mỗi lần, một nhóm quan sát khác nhau được xử lý. như một tập dữ liệu kiểm định. Quá trình này tạo ra k ước tính về sai số kiểm định. $MSE_1, MSE_2, \dots, MSE_k$. Ước tính CV k-fold được tính bằng cách lấy trung bình những giá trị này,

CV k lần

$$CV(k) = \frac{1}{k} \sum_{i=1}^k MSE_i. \tag{5.3}$$

Hình 5.5 minh họa phương pháp kiểm định chéo k lần.

¹Trong trường hợp hồi quy tuyến tính đa biến, đòn bẩy có dạng phức tạp hơn một chút so với (3.37), nhưng (5.2) vẫn đúng.

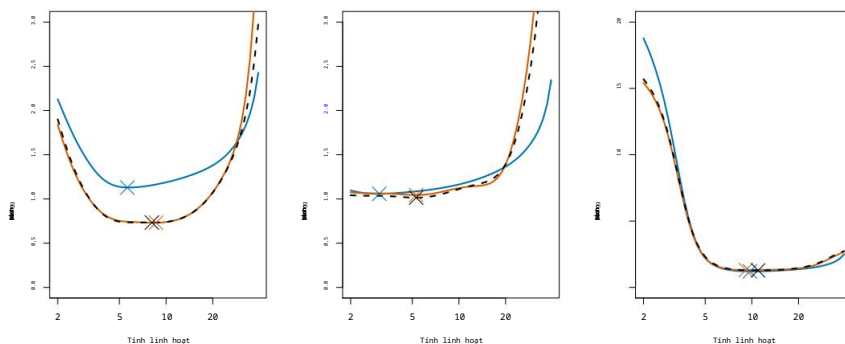


HÌNH 5.5. Sơ đồ minh họa phương pháp kiểm định chéo 5 lần (5-fold CV). Một tập hợp gồm n quan sát được chia ngẫu nhiên thành năm nhóm không chồng chéo. Mỗi nhóm năm phần này đóng vai trò là tập dữ liệu kiểm định (màu be), và phần còn lại là tập dữ liệu huấn luyện (màu xanh lam). Sai số kiểm định được ước tính bằng cách lấy trung bình năm ước tính MSE thu được.

Không khó để nhận thấy rằng LOOCV là trường hợp đặc biệt của k -fold CV trong đó k được đặt bằng n . Trên thực tế, người ta thường thực hiện k -fold CV bằng cách sử dụng $k = 5$ hoặc $k = 10$. Ưu điểm của việc sử dụng $k = 5$ hoặc $k = 10$ thay vì $k = n$ là gì? Ưu điểm rõ ràng nhất là về mặt tính toán. LOOCV yêu cầu huấn luyện phương pháp học thống kê n lần. Điều này có khả năng tốn kém về mặt tính toán (ngoại trừ các mô hình tuyến tính được huấn luyện bằng phương pháp bình phương nhỏ nhất, trong trường hợp đó công thức (5.2) có thể được sử dụng). Nhưng kiểm định chéo là một phương pháp rất tổng quát có thể được áp dụng cho hầu hết mọi phương pháp học thống kê. Một số phương pháp học thống kê có quy trình huấn luyện tốn nhiều tài nguyên tính toán, do đó việc thực hiện LOOCV có thể gây ra các vấn đề về tính toán, đặc biệt nếu n rất lớn. Ngược lại, việc thực hiện 10-fold CV chỉ yêu cầu huấn luyện quy trình học mười lần, điều này có thể khả thi hơn nhiều. Như chúng ta thấy trong Phần 5.1.4, việc thực hiện kiểm định chéo 5 lần hoặc 10 lần cũng có thể mang lại những lợi ích phi tính toán khác, liên quan đến sự đánh đổi giữa độ lệch và phương sai.

Hình 5.4, bảng bên phải, hiển thị chín ước tính CV 10 lần khác nhau cho tập dữ liệu *Auto*, mỗi ước tính thu được từ một cách chia ngẫu nhiên các quan sát thành mười phần khác nhau. Như chúng ta có thể thấy từ hình, có một số biến động trong các ước tính CV do sự biến động trong cách chia các quan sát thành mười phần. Nhưng sự biến động này thường thấp hơn nhiều so với sự biến động trong các ước tính lỗi kiểm thử thu được từ phương pháp tập dữ liệu xác thực (bảng bên phải của Hình 5.2).

Khi xem xét dữ liệu thực tế, chúng ta không biết MSE kiểm định thực sự, do đó khó xác định độ chính xác của ước tính kiểm định chéo. Tuy nhiên, nếu chúng ta xem xét dữ liệu mô phỏng, thì chúng ta có thể tính toán MSE kiểm tra thực sự, và từ đó có thể đánh giá độ chính xác của kết quả kiểm định chéo. Trong Hình 5.6, chúng ta vẽ biểu đồ ước tính kiểm định chéo và tỷ lệ lỗi kiểm tra thực sự thu được từ việc áp dụng spline làm mịn cho các tập dữ liệu mô phỏng được minh họa trong Hình 2.9-2.11 của Chương 2. MSE kiểm tra thực sự được hiển thị bằng màu xanh lam. Các đường chấm đen và đường liền màu cam lần lượt hiển thị các ước tính LOOCV và CV 10 lần. Trong cả ba biểu đồ, hai ước tính kiểm định chéo rất giống nhau. Trong bảng bên phải



HÌNH 5.6. MSE kiểm tra thực và ước tính cho các tập dữ liệu mô phỏng trong

Hình 2.9 (trái), 2.10 (giữa) và 2.11 (phải). Sai số bình phương trung bình (MSE) thực tế của bài kiểm tra được hiển thị. Trong bảng màu xanh lam, ước tính LOOCV được hiển thị dưới dạng đường nét đứt màu đen, và phương pháp kiểm định chéo 10 lần. Giá trị ước tính CV được hiển thị bằng màu cam. Các dấu chéo biểu thị giá trị nhỏ nhất của mỗi yếu tố các đường cong MSE.

Như trong Hình 5.6, sai số bình phương trung bình (MSE) của bài kiểm tra thực và đường cong kiểm định chéo gần như trùng khớp.

Giống hệt nhau. Ở bảng giữa của Hình 5.6, hai tập hợp đường cong tương tự nhau.

Ở mức độ linh hoạt thấp hơn, trong khi đường cong CV đánh giá quá cao kết quả kiểm tra.

Đặt MSE để có mức độ linh hoạt cao hơn. Trong bảng bên trái của Hình 5.6,

Các đường cong CV có hình dạng tổng thể chính xác, nhưng chúng đánh giá thấp hơn thực tế.

Kiểm tra thực tế MSE.

Khi thực hiện kiểm định chéo, mục tiêu của chúng ta có thể là xác định cách thức

Một quy trình học thống kê nhất định có thể được kỳ vọng sẽ hoạt động như thế nào trên

dữ liệu độc lập; trong trường hợp này, ước tính thực tế của MSE kiểm định là

có thể gây hứng thú. Nhưng đôi khi chúng ta chỉ quan tâm đến vị trí của...

Điểm cực tiểu trong đường cong MSE kiểm định ước tính. Điều này là do chúng ta

có thể đang thực hiện kiểm định chéo trên một số thuật toán học thống kê.

các phương pháp, hoặc trên một phương pháp duy nhất sử dụng các mức độ linh hoạt khác nhau, theo thứ tự

Để xác định phương pháp nào dẫn đến sai số kiểm thử thấp nhất. Vì mục đích này,

Vị trí của điểm cực tiểu trên đường cong MSE kiểm định ước tính là

quan trọng, nhưng giá trị thực tế của MSE kiểm định ước tính thì không. Chúng tôi thấy

trong Hình 5.6 cho thấy mặc dù đôi khi họ đánh giá thấp

MSE kiểm tra thực sự, tất cả các đường cong CV đều gần như xác định được giá trị chính xác.

mức độ linh hoạt—tức là mức độ linh hoạt tương ứng với mức nhỏ nhất

Kiểm tra MSE.

5.1.4 Sự đánh đổi giữa độ lệch và phương sai trong phương pháp kiểm định chéo k-fold

Chúng tôi đã đề cập trong Mục 5.1.3 rằng phương pháp kiểm định chéo k lần với $k < n$ có lợi thế về mặt tính toán so với phương pháp kiểm định chéo một lần duy nhất (LOOCV). Nhưng bỏ qua các vấn đề về tính toán,

Một ưu điểm ít rõ ràng hơn nhưng có khả năng quan trọng hơn của phương pháp kiểm định chéo k lần là

Điều đó thường đưa ra ước tính chính xác hơn về tỷ lệ lỗi kiểm tra so với các phương pháp khác.

LOOCV. Điều này liên quan đến sự đánh đổi giữa độ lệch và phương sai.

Như đã đề cập trong Mục 5.1.1, phương pháp tập dữ liệu xác thực có thể

dẫn đến việc đánh giá quá cao tỷ lệ lỗi kiểm tra, vì trong phương pháp này

Tập dữ liệu huấn luyện được sử dụng để huấn luyện phương pháp học thống kê chỉ chứa một nửa số dữ liệu đó.

các quan sát từ toàn bộ tập dữ liệu. Sử dụng logic này, không khó để thấy rằng...

Phương pháp LOOCV sẽ cho ước lượng sai số kiểm định gần như không thiên lệch, vì mỗi tập huấn luyện chứa $n - 1$ quan sát, gần bằng số lượng quan sát trong tập dữ liệu đầy đủ. Và việc thực hiện k-fold CV, ví dụ với $k = 5$ hoặc $k = 10$, sẽ dẫn đến mức độ thiên lệch trung gian, vì mỗi tập huấn luyện chứa khoảng $(k - 1)n/k$ quan sát – ít hơn so với phương pháp LOOCV, nhưng nhiều hơn đáng kể so với phương pháp tập dữ liệu xác thực. Do đó, từ góc độ giảm thiểu thiên lệch, rõ ràng là phương pháp LOOCV được ưu tiên hơn so với k-fold CV.

Tuy nhiên, chúng ta biết rằng sai lệch không phải là nguồn gây lo ngại duy nhất trong một quy trình ước lượng; chúng ta cũng phải xem xét phương sai của quy trình đó. Thực tế cho thấy LOOCV có phương sai cao hơn so với k-fold CV với $k < n$. Tại sao lại như vậy? Khi thực hiện LOOCV, về cơ bản chúng ta đang lấy trung bình các kết quả đầu ra của n mô hình đã được hiệu chỉnh, mỗi mô hình được huấn luyện trên một tập hợp quan sát gần như giống hệt nhau; do đó, các kết quả đầu ra này có mối tương quan cao (tích cực) với nhau. Ngược lại, khi thực hiện k-fold CV với $k < n$, chúng ta đang lấy trung bình các kết quả đầu ra của k mô hình đã được hiệu chỉnh có mối tương quan thấp hơn với nhau, vì sự chồng chéo giữa các tập huấn luyện trong mỗi mô hình nhỏ hơn. Vì trung bình của nhiều đại lượng có mối tương quan cao có phương sai cao hơn so với trung bình của nhiều đại lượng có mối tương quan thấp hơn, nên ước lượng sai số kiểm định thu được từ LOOCV có xu hướng có phương sai cao hơn so với ước lượng sai số kiểm định thu được từ k-fold CV.

Tóm lại, có sự đánh đổi giữa độ lệch và phương sai liên quan đến việc lựa chọn k trong phương pháp kiểm định chéo k-fold. Thông thường, dựa trên những cân nhắc này, người ta thực hiện kiểm định chéo k-fold bằng cách sử dụng $k = 5$ hoặc $k = 10$, vì các giá trị này đã được chứng minh bằng thực nghiệm là cho ra ước tính tỷ lệ lỗi kiểm thử không bị độ lệch quá cao cũng không bị phương sai quá cao.

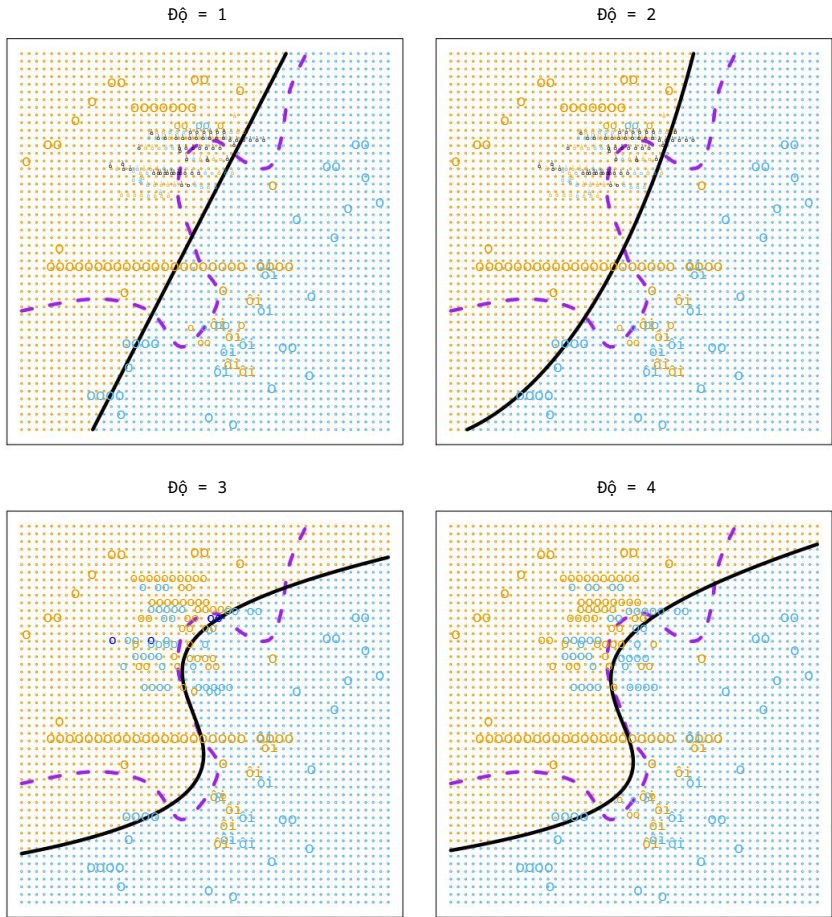
5.1.5 Kiểm định chéo trên các bài toán phân loại

Trong chương này, chúng ta đã minh họa việc sử dụng phương pháp kiểm định chéo trong mô hình hồi quy khi kết quả Y là biến định lượng, và do đó đã sử dụng MSE để định lượng sai số kiểm định. Tuy nhiên, kiểm định chéo cũng có thể là một phương pháp rất hữu ích trong mô hình phân loại khi Y là biến định tính. Trong trường hợp này, kiểm định chéo hoạt động giống như đã mô tả trước đó trong chương này, ngoại trừ việc thay vì sử dụng MSE để định lượng sai số kiểm định, chúng ta sử dụng số lượng quan sát bị phân loại sai. Ví dụ, trong mô hình phân loại, tỷ lệ lỗi LOOCV có dạng như sau:

$$CV(n) = \frac{1}{N} \sum_{i=1}^N \text{Erri}, \quad (5.4)$$

trong đó $\text{Erri} = I(y_i \neq \hat{y}_i)$. Tỷ lệ lỗi CV k-fold và tỷ lệ lỗi của tập dữ liệu xác thực được định nghĩa tương tự.

Ví dụ, chúng tôi đã áp dụng nhiều mô hình hồi quy logistic khác nhau cho dữ liệu phân loại hai chiều được hiển thị trong Hình 2.13. Trong bảng phía trên bên trái của Hình 5.7, đường liền nét màu đen thể hiện ranh giới quyết định ước tính thu được từ việc áp dụng mô hình hồi quy logistic chuẩn cho tập dữ liệu này. Vì đây là dữ liệu mô phỏng, chúng ta có thể tính toán tỷ lệ lỗi kiểm thử thực tế, có giá trị là 0,201 và do đó lớn hơn đáng kể so với giá trị Bayes.



HÌNH 5.7. Đường cong hồi quy logistic phù hợp với dữ liệu phân loại hai chiều được hiển thị trong Hình 2.13. Đường ranh giới quyết định Bayes được biểu thị bằng đường chấm tím. Các đường ranh giới quyết định ước tính từ hồi quy logistic tuyến tính, bậc hai, bậc ba và bậc bốn (bậc 1-4) được hiển thị bằng màu đen. Tỷ lệ lỗi kiểm định cho bốn đường cong hồi quy logistic phù hợp lần lượt là 0,201, 0,197, 0,160 và 0,162, trong khi tỷ lệ lỗi Bayes là 0,133.

Tỷ lệ lỗi là 0,133. Rõ ràng hồi quy logistic không đủ linh hoạt để mô hình hóa ranh giới quyết định Bayes trong thiết lập này. Chúng ta có thể dễ dàng mở rộng hồi quy logistic để thu được ranh giới quyết định phi tuyến tính bằng cách sử dụng các hàm đa thức của các biến dự báo, như chúng ta đã làm trong thiết lập hồi quy ở Phần 3.3.2. Ví dụ, chúng ta có thể phù hợp với mô hình hồi quy logistic bậc hai , được cho bởi

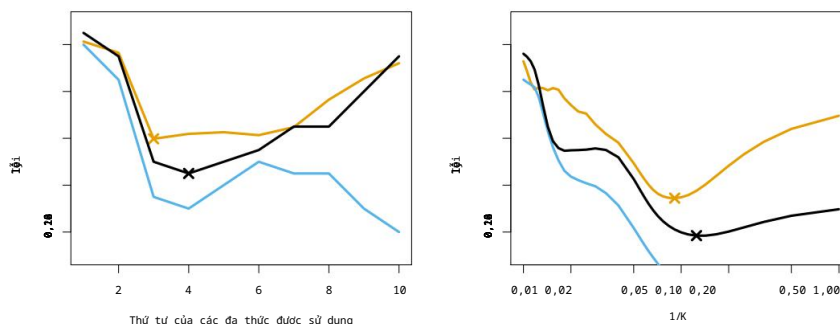
nhất ký

$$\frac{P}{1 + p}$$

$$= \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2$$

2 . (5.5)

Hình 5.7 , bảng phía trên bên phải , hiển thị ranh giới quyết định thu được, giờ đây đã cong. Tuy nhiên, tỷ lệ lỗi kiểm thử chỉ được cải thiện một chút, xuống còn 0,197. Sự cải thiện đáng kể hơn nhiều được thể hiện rõ ở bảng phía dưới bên trái.



HÌNH 5.8. Lỗi kiểm thử (màu nâu), lỗi huấn luyện (màu xanh lam) và lỗi kiểm định chéo 10 lần. (màu đen) trên dữ liệu phân loại hai chiều được hiển thị trong Hình 5.7. Bên trái: Hồi quy logistic sử dụng các hàm đa thức của các biến dự báo. Thứ tự của Các đa thức được sử dụng được hiển thị trên trục x. Bên phải: Bộ phân loại KNN với Các giá trị khác nhau của K, số lượng láng giềng được sử dụng trong bộ phân loại KNN.

Hình 5.7, trong đó chúng tôi đã áp dụng mô hình hồi quy logistic liên quan đến đa thức bậc ba của các biến dự đoán. Giờ đây, tỷ lệ lỗi kiểm thử đã giảm. Đến 0.160. Việc chuyển sang đa thức bậc bốn (phía dưới bên phải) làm tăng nhẹ Lỗi kiểm tra.

Trên thực tế, với dữ liệu thực, ranh giới quyết định Bayes và tỷ lệ lỗi kiểm thử là không xác định. Vậy làm thế nào chúng ta có thể lựa chọn giữa bốn mô hình logistic? Các mô hình hồi quy được hiển thị trong Hình 5.7? Chúng ta có thể sử dụng phương pháp kiểm định chéo trong để đưa ra quyết định này. Bảng bên trái của Hình 5.8 hiển thị trong Biểu đồ này thể hiện tỷ lệ lỗi CV gấp 10 lần do việc áp dụng mười mô hình hồi quy logistic vào dữ liệu, sử dụng các hàm đa thức của các biến dự báo. đến bậc thứ mười. Các lỗi kiểm thử thực tế được hiển thị bằng màu nâu, và quá trình huấn luyện Các lỗi được hiển thị bằng màu xanh lam. Như chúng ta đã thấy trước đây, lỗi huấn luyện có xu hướng giảm khi độ linh hoạt của sự vừa vặn tăng lên. (Hình minh họa cho thấy) Mặc dù tỷ lệ lỗi trong quá trình huấn luyện không giảm dần một cách đơn điệu, (Nhìn chung, nó có xu hướng giảm khi độ phức tạp của mô hình tăng lên.) Ngược lại, sai số kiểm tra hiển thị hình chữ U đặc trưng. Hệ số biến thiên 10 lần (10-fold CV) Tỷ lệ lỗi cung cấp một sự ước tính khá tốt về tỷ lệ lỗi của bài kiểm tra. Mặc dù nó phần nào đánh giá thấp tỷ lệ lỗi, nhưng nó đạt đến mức tối thiểu. khi sử dụng đa thức bậc bốn, giá trị này rất gần với điểm cực tiểu của đường cong thử nghiệm, điều này xảy ra khi sử dụng đa thức bậc ba. đã được sử dụng. Trên thực tế, việc sử dụng đa thức bậc bốn có thể dẫn đến kết quả kiểm tra tốt. hiệu suất được thiết lập, vì tỷ lệ lỗi kiểm tra thực tế gần như giống nhau đối với Đa thức bậc ba, bậc bốn, bậc năm và bậc sáu.

Hình 5.8, bảng bên phải, hiển thị ba đường cong tương tự bằng cách sử dụng phương pháp KNN để phân loại, như một hàm của giá trị K.

(trong ngữ cảnh này, điều đó biểu thị số lượng láng giềng được sử dụng trong thuật toán KNN) bộ phân loại, chứ không phải số lượng các fold CV được sử dụng). Một lần nữa, quá trình huấn luyện Tỷ lệ lỗi giảm khi phương pháp trở nên linh hoạt hơn, và do đó chúng ta thấy rằng

Tỷ lệ lỗi huấn luyện không thể được sử dụng để chọn giá trị tối ưu cho K.

Mặc dù đường cong lỗi kiểm định chéo hơi đánh giá thấp kết quả kiểm định.

Tỷ lệ lỗi, nó đạt mức tối thiểu rất gần với giá trị tối ưu của K.

5.2 Phương pháp Bootstrap

Phương pháp bootstrap là một công cụ thống kê mạnh mẽ và được ứng dụng rộng rãi, có thể được sử dụng để định lượng sự không chắc chắn liên quan đến một ước lượng hoặc phương pháp học thống kê nhất định. Ví dụ đơn giản là phương pháp bootstrap. khởi động có thể được sử dụng để ước tính sai số chuẩn của các hệ số từ một phương trình tuyến tính.

sự phù hợp của hồi quy. Trong trường hợp cụ thể của hồi quy tuyến tính, điều này không đặc biệt quan trọng. hữu ích, vì chúng ta đã thấy trong Chương 3 rằng các phần mềm thống kê tiêu chuẩn như R tự động xuất ra các sai số chuẩn như vậy. Tuy nhiên, sức mạnh của Ưu điểm của Bootstrap nằm ở chỗ nó có thể dễ dàng áp dụng cho nhiều lĩnh vực khác nhau. các phương pháp học thống kê, bao gồm một số phương pháp mà việc đo lường độ biến thiên rất khó khăn và không được tự động xuất ra bởi phần mềm thống kê.

Trong phần này, chúng tôi minh họa quá trình bootstrap bằng một ví dụ đơn giản, trong đó chúng ta... Tôi muốn xác định phương án phân bổ đầu tư tối ưu nhất theo một mô hình đơn giản. Trong Phần 5.3, chúng ta sẽ tìm hiểu việc sử dụng phương pháp bootstrap để đánh giá tính biến thiên. liên quan đến các hệ số hồi quy trong mô hình tuyến tính đã được hiệu chỉnh.

Giả sử chúng ta muốn đầu tư một khoản tiền cố định vào hai lĩnh vực tài chính. các tài sản mang lại lợi nhuận lần lượt là X và Y, trong đó X và Y là các lượng ngẫu nhiên. Chúng ta sẽ đầu tư một phần α tiền của mình vào X, và sẽ Đầu tư số tiền còn lại 1 - α vào Y. Vì có sự biến động liên quan đến Với lợi nhuận thu được từ hai tài sản này, chúng ta muốn chọn α sao cho tổng lợi nhuận là nhỏ nhất. rủi ro, hay sự biến động, của khoản đầu tư của chúng ta. Nói cách khác, chúng ta muốn giảm thiểu rủi ro. Var(αX + (1 - α)Y). Có thể chứng minh rằng giá trị làm giảm thiểu rủi ro được đưa ra bởi

α = (σ²_Y - σ_XY) / (σ²_X + σ²_Y - 2σ_XY), (5.6)

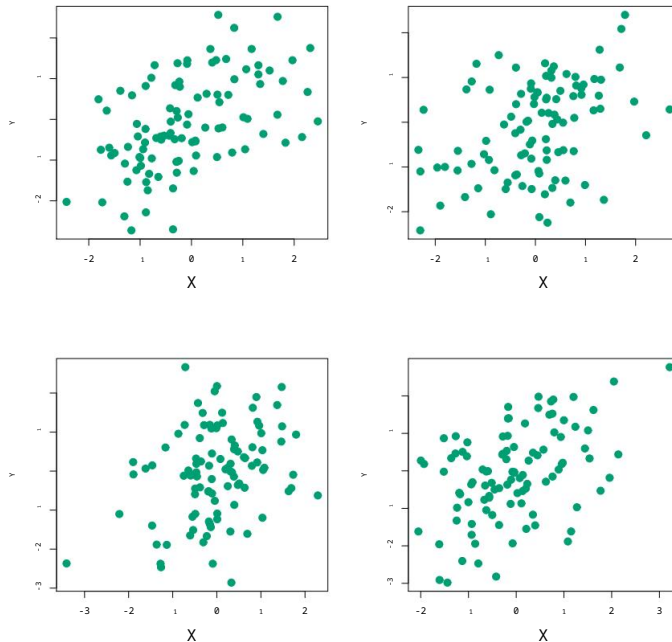
trong đó σ²_X = Var(X), σ²_Y = Var(Y), và σ_XY = Cov(X, Y).

Trên thực tế, các đại lượng σ²_X, σ²_Y, và σ_XY chưa biết. Chúng ta có thể tính toán ước tính cho các đại lượng này, σ²_X, σ²_Y, và σ_XY, sử dụng một tập dữ liệu mà Chứa các phép đo trước đây cho X và Y. Sau đó, chúng ta có thể ước tính giá trị. của α sao cho giảm thiểu phương sai của khoản đầu tư của chúng ta bằng cách sử dụng

α̂ = (σ̂²_Y - σ̂_XY) / (σ̂²_X + σ̂²_Y - 2σ̂_XY). (5.7)

Hình 5.9 minh họa phương pháp này để ước tính α trên dữ liệu mô phỏng. tập hợp. Trong mỗi bảng, chúng tôi đã mô phỏng 100 cặp lợi nhuận cho các khoản đầu tư. X và Y. Chúng tôi đã sử dụng các lợi nhuận này để ước tính σ²_X, σ²_Y, và σ_XY, mà chúng ta sau đó thay vào (5.7) để có được ước tính cho α. Giá trị của Giá trị α̂ thu được từ mỗi bộ dữ liệu mô phỏng nằm trong khoảng từ 0,532 đến 0,657.

Việc muốn định lượng độ chính xác của ước tính α của chúng ta là điều tự nhiên. ước tính độ lệch chuẩn của α̂, chúng tôi đã lặp lại quá trình mô phỏng 100 cặp quan sát của X và Y và ước tính α bằng cách sử dụng (5.7), 1.000 lần. Nhờ đó, chúng ta thu được 1.000 ước tính cho α, mà chúng ta có thể gọi là α¹, α²,..., α¹,000. Bảng bên trái của Hình 5.10 hiển thị biểu đồ tần số của các ước tính thu được. Đối với các mô phỏng này, các tham số được đặt là σ²_X = 1, σ²_Y = 1,25 và σ_XY = 0,5, vì vậy chúng ta biết rằng giá trị thực của α có giá trị là 0,6. Chúng tôi đã biểu thị giá trị này bằng một đường thẳng đứng liên nét trên biểu đồ tần số.



HÌNH 5.9. Mỗi bảng hiển thị 100 kết quả đầu tư mô phỏng.

X và Y. Từ trái sang phải và từ trên xuống dưới, các ước tính thu được cho α .

Các giá trị đó lần lượt là 0,576, 0,532, 0,657 và 0,651.

Giá trị trung bình trên tất cả 1.000 ước tính cho α là

$$\bar{\alpha} = \frac{1}{1000} \sum_{i=1}^{1000} \alpha_i^* = 0,5996,$$

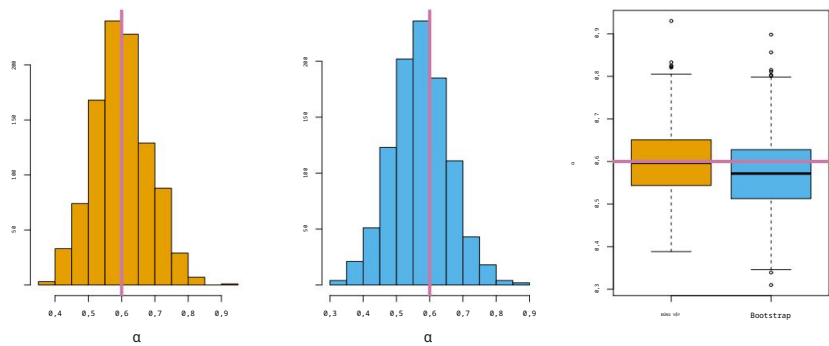
rất gần với $\alpha = 0,6$, và độ lệch chuẩn của các ước tính là

$$s_{\bar{\alpha}} = \frac{1}{1000} \sqrt{\sum_{i=1}^{1000} (\alpha_i^* - \bar{\alpha})^2} = 0,083.$$

Điều này cho chúng ta một cái nhìn rất tốt về độ chính xác của $\bar{\alpha}$: $SE(\bar{\alpha}) \approx 0,083$. Vì vậy Nói một cách đại khái, đối với một mẫu ngẫu nhiên từ quần thể, chúng ta sẽ Trung bình, ta kỳ vọng $\bar{\alpha}$ sẽ khác α khoảng 0,08.

Tuy nhiên, trên thực tế, quy trình ước tính $SE(\bar{\alpha})$ được nêu ở trên vẫn được áp dụng. Không thể áp dụng được, vì với dữ liệu thực tế, chúng ta không thể tạo ra các mẫu mới từ quần thể ban đầu. Tuy nhiên, phương pháp bootstrap cho phép chúng ta Sử dụng máy tính để mô phỏng quá trình thu thập các bộ mẫu mới. để chúng ta có thể ước tính sự biến thiên của $\bar{\alpha}$ mà không cần tạo thêm dữ liệu. các mẫu. Thay vì liên tục thu thập các bộ dữ liệu độc lập từ... Thay vào đó, chúng ta thu được các tập dữ liệu riêng biệt bằng cách lấy mẫu lặp đi lặp lại. Các quan sát từ tập dữ liệu gốc.

Cách tiếp cận này được minh họa trong Hình 5.11 trên một tập dữ liệu đơn giản, trong đó chúng ta gọi Z là tập hợp chỉ chứa $n = 3$ quan sát. Chúng ta chọn ngẫu nhiên n Các quan sát từ tập dữ liệu được sử dụng để tạo ra tập dữ liệu bootstrap.



HÌNH 5.10. Bên trái: Biểu đồ tần suất của các ước tính α thu được bằng cách tạo ra 1.000 bộ dữ liệu mô phỏng từ quần thể thực. Trung tâm: Biểu đồ tần suất của... Các ước tính về α thu được từ 1.000 mẫu bootstrap từ một tập dữ liệu duy nhất. Bên phải: Các ước tính của α được hiển thị ở bảng bên trái và bảng giữa được thể hiện như sau: Biểu đồ hộp. Trong mỗi bảng, đường màu hồng biểu thị giá trị thực của α .

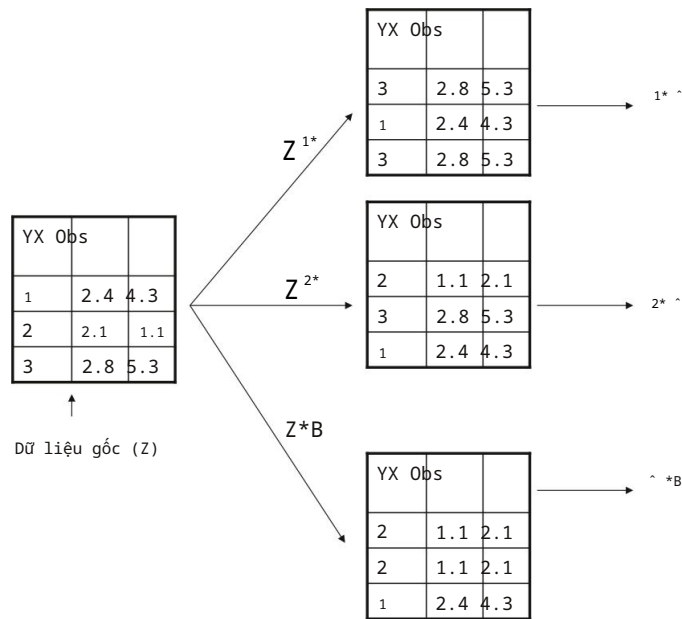
Z 1. Việc lấy mẫu được thực hiện có hoàn trả, có nghĩa là cùng một quan sát có thể xuất hiện nhiều hơn một lần trong tập dữ liệu bootstrap. Trong ví dụ này, Z 1 chứa quan sát thứ ba hai lần, quan sát đầu tiên. Trong ví dụ này, Z 1 chứa quan sát thứ ba hai lần, quan sát đầu tiên. một lần, và không có trường hợp quan sát thứ hai nào. Lưu ý rằng nếu một quan sát Nếu Z*1 nằm trong Z*1, thì cả giá trị X và Y của nó đều được bao gồm. Ta có thể sử dụng Z 1 để tạo ra một ước tính bootstrap mới cho α , mà chúng ta gọi là $\hat{\alpha}^1$. Điều này Quy trình được lặp lại B lần với một giá trị B lớn nào đó, nhằm tạo ra B là các bộ dữ liệu bootstrap khác nhau, Z 1, Z 2, ..., Z B, và B là các giá trị α tương ứng. ước tính, $\hat{\alpha}^1, \hat{\alpha}^2, \dots, \hat{\alpha}^B$. Chúng ta có thể tính sai số chuẩn của những ước tính này. ước tính bootstrap bằng cách sử dụng công thức

với
thay thế

$$SEB(\hat{\alpha}) = \sqrt{\frac{1}{B} \sum_{r=1}^B (\hat{\alpha}^r - \bar{\alpha})^2} \quad (5.8)$$

Điều này đóng vai trò như một ước tính về sai số chuẩn của $\hat{\alpha}$ được ước tính từ Bộ dữ liệu gốc.

Phương pháp bootstrap được minh họa ở phần giữa của Hình 5.10. Biểu đồ này hiển thị biểu đồ tần suất của 1.000 ước tính bootstrap của α , mỗi ước tính được tính toán bằng cách sử dụng một tập dữ liệu bootstrap riêng biệt. Biểu đồ này được xây dựng dựa trên... Nó dựa trên một tập dữ liệu duy nhất, và do đó có thể được tạo ra bằng cách sử dụng dữ liệu thực. Lưu ý rằng biểu đồ tần số trông rất giống với biểu đồ bên trái, điều này Biểu đồ này hiển thị biểu đồ tần suất lý tưởng của các ước tính α thu được bằng cách tạo ra 1.000 bộ dữ liệu mô phỏng từ quần thể thực. Cụ thể, Ước tính bootstrap $SE(\hat{\alpha})$ từ (5.8) là 0,087, rất gần với ước tính của Giá trị 0,083 thu được bằng cách sử dụng 1.000 bộ dữ liệu mô phỏng. Bảng bên phải hiển thị thông tin ở bảng giữa và bảng bên trái theo một cách khác, thông qua Biểu đồ hộp của các ước tính cho α thu được bằng cách tạo ra 1.000 dữ liệu mô phỏng. các tập hợp từ quần thể thực và sử dụng phương pháp bootstrap. Một lần nữa, Biểu đồ hộp có độ phân tán tương tự, cho thấy phương pháp bootstrap có thể có thể được sử dụng để ước tính hiệu quả sự biến thiên liên quan đến α .



HÌNH 5.11. Minh họa đồ họa về phương pháp bootstrap trên một mô hình nhỏ. Mẫu chứa $n = 3$ quan sát. Mỗi tập dữ liệu bootstrap chứa n quan sát, được lấy mẫu có hoàn lại từ tập dữ liệu gốc. Mỗi tập dữ liệu bootstrap Tập hợp này được sử dụng để ước tính giá trị của α .

5.3 Bài thực hành: Kiểm định chéo và phương pháp Bootstrap

Trong bài thực hành này, chúng ta sẽ khám phá các kỹ thuật lấy mẫu lại được đề cập trong chương này. Một số lệnh trong bài thực hành này có thể mất một thời gian để chạy trên máy của bạn. máy tính.

Chúng tôi lại bắt đầu bằng cách đặt hầu hết các mặt hàng nhập khẩu của mình ở cấp độ cao nhất này.

```
Trong [1]: import numpy as np
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                          Tóm lại,
                          đa)
from sklearn.model_selection import train_test_split
```

Phòng thí nghiệm này cần nhập khẩu một số thiết bị mới.

```
Trong [2]: từ functools nhập một phần
from sklearn.model_selection import \
    (kiểm tra chéo,
     KFold,
     ShuffleSplit)
from sklearn.base import clone
from ISLP.models import sklearn_sm
```

5.3.1 Phương pháp tập dữ liệu kiểm định

Chúng tôi nghiên cứu việc sử dụng phương pháp tập dữ liệu kiểm định để ước tính tỷ lệ lỗi kiểm thử phát sinh từ việc áp dụng các mô hình tuyến tính khác nhau trên tập dữ liệu `Auto`.

Chúng ta sử dụng hàm `train_test_split()` để chia dữ liệu thành tập huấn luyện và tập kiểm tra. Vì có 392 quan sát, chúng ta chia thành hai tập bằng nhau có kích thước 196 bằng cách sử dụng đối số `test_size=196`. Nói chung, nên đặt một hạt giống ngẫu nhiên khi thực hiện các thao tác như thế này có chứa yếu tố ngẫu nhiên, để kết quả thu được có thể được tái tạo chính xác vào thời điểm sau đó. Chúng ta đặt hạt giống ngẫu nhiên của bộ chia bằng đối số `random_state=0`.

`train_test_split()`

```
Trong [3]: Auto = load_data('Auto')
Auto_train, Auto_valid = train_test_split(Auto,
                                          test_size=196,
                                          random_state=0)
```

Giờ đây, chúng ta có thể thực hiện hồi quy tuyến tính chỉ sử dụng các quan sát tương ứng với tập dữ liệu huấn luyện `Auto_train`.

```
Trong [4]: hp_mm = MS(['horsepower'])
X_train = hp_mm.fit_transform(Auto_train) y_train =
Auto_train['mpg'] model =
sm.OLS(y_train, X_train) results =
model.fit()
```

Chúng ta sử dụng phương thức `predict()` để đánh giá kết quả trên ma trận mô hình cho mô hình này được tạo ra bằng tập dữ liệu xác thực. Chúng ta cũng tính toán MSE xác thực của mô hình.

```
Trong [5]: X_valid = hp_mm.transform(Auto_valid) y_valid
= Auto_valid['mpg'] valid_pred =
results.predict(X_valid) np.mean((y_valid -
valid_pred)**2)
```

Ra[5]: 23.6166

Do đó, ước tính của chúng tôi về MSE kiểm định của mô hình hồi quy tuyến tính là 23,62.

Chúng ta cũng có thể ước tính sai số xác thực cho các hồi quy đa thức bậc cao hơn. Trước tiên, chúng tôi cung cấp một hàm `evalMSE()` nhận vào một chuỗi mô hình cũng như một tập huấn luyện và một tập kiểm tra và trả về MSE trên tập kiểm tra.

```
Trong [6]: def evalMSE(terms,
                    phản ứng,
                    huấn luyện,
                    kiểm tra):

    mm = MS(terms)
    X_train = mm.fit_transform(train) y_train =
    train[response]

    X_test = mm.transform(test) y_test
    = test[response]
```



```
results = sm.OLS(y_train, X_train).fit()
test_pred = results.predict(X_test)

return np.mean((y_test - test_pred)**2)
```

Chúng ta hãy sử dụng hàm này để ước tính MSE xác thực bằng cách sử dụng các phép khớp tuyến tính, bậc hai và bậc ba. Ở đây, chúng ta sử dụng hàm `enumerate()`, hàm này cung cấp cả giá trị và chỉ số của các đối tượng khi lặp qua vòng lặp `for`.

liệt kê()

```
Trong [7]: MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)], 'mpg',

                        Tự động huấn luyện,
                        Tự động xác thực)

MSE
```

```
Out[7]: array([23.62, 18.76, 18.80])
```

Tỷ lệ lỗi này lần lượt là 23,62%, 18,76% và 18,80%. Nếu chúng ta chọn cách phân chia tập huấn luyện/kiểm chứng khác, thì ta có thể kỳ vọng tỷ lệ lỗi trên tập kiểm chứng sẽ khác nhau.

```
Trong [8]: Auto_train, Auto_valid = train_test_split(Auto,
                                                    test_size=196,
                                                    random_state=3)

MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
    MSE[idx] = evalMSE([poly('horsepower', degree)], 'mpg',

                        Tự động huấn luyện,
                        Tự động xác thực)

MSE
```

```
Out[8]: array([20.76, 16.95, 16.97])
```

Bằng cách chia các quan sát thành tập huấn luyện và tập kiểm chứng, chúng ta thấy rằng tỷ lệ lỗi của tập kiểm chứng đối với các mô hình có các số hạng tuyến tính, bậc hai và bậc ba lần lượt là 20,76%, 16,95% và 16,97%.

Những kết quả này phù hợp với các phát hiện trước đây của chúng tôi: mô hình dự đoán `mpg` sử dụng hàm bậc hai của `mã lực` hoạt động tốt hơn mô hình chỉ sử dụng hàm tuyến tính của `mã lực`, và không có bằng chứng nào cho thấy sự cải thiện khi sử dụng hàm bậc ba của `mã lực`.

5.3.2 Kiểm định chéo

Về lý thuyết, ước tính kiểm định chéo có thể được tính toán cho bất kỳ mô hình tuyến tính tổng quát nào. Tuy nhiên, trên thực tế, cách đơn giản nhất để kiểm định chéo trong Python là sử dụng `sklearn`, thư viện này có giao diện hoặc API khác với `statsmodels`, đoạn mã mà chúng ta đã sử dụng để huấn luyện các mô hình GLM.

Đây là một vấn đề thường gặp đối với các nhà khoa học dữ liệu: “Tôi có một hàm để thực hiện nhiệm vụ A, và cần đưa nó vào một hàm khác thực hiện nhiệm vụ B, để tôi có thể tính toán $B(A(D))$, trong đó D là dữ liệu của tôi.” Khi A và B không tương tác trực tiếp với nhau, điều này đòi hỏi phải sử dụng một lớp bao bọc (wrapper). Trong `ISLP`,

bao bì

Trong gói này, chúng tôi cung cấp một hàm bao bọc, `sklearn_sm()`, cho phép chúng ta dễ dàng sử dụng các công cụ kiểm định chéo của `sklearn` với các mô hình được huấn luyện bởi `statsmodels`. `sklearn_sm()`

Lớp `sklearn_sm()` có đối số đầu tiên là một mô hình từ `statsmodels`. Hàm này có thể nhận thêm hai đối số tùy chọn: `model_str` dùng để chỉ định công thức, và `model_args` là một từ điển chứa các đối số bổ sung được sử dụng khi huấn luyện mô hình. Ví dụ, để huấn luyện mô hình hồi quy logistic, chúng ta phải chỉ định đối số `family`. Đối số này được truyền dưới dạng `model_args={'family':sm.families.Binomial()}`.

Đây là cách mà trình bao bọc của chúng tôi hoạt động:

```
Trong [9]: hp_model = sklearn_sm(sm.OLS,
                                MS(['horsepower']))
X, Y = Auto.drop(columns=['mpg'], Auto['mpg']) cv_results =
cross_validate(hp_model, X, Y, cv=Auto.shape[0])

cv_err =
np.mean(cv_results['test_score'])
cv_err
```

Ra ngoài[9]: 24.2315

Các đối số của hàm `cross_validate()` như sau: một đối tượng với các phương thức `fit()`, `predict()` và `score()` phù hợp, một mảng các đặc trưng `X` và một biến phản hồi `Y`. Chúng tôi cũng đã thêm một đối số `cv` vào hàm `cross_validate()`; việc chỉ định một số nguyên `K` sẽ dẫn đến kiểm định chéo `K-fold`. Chúng tôi đã cung cấp một giá trị tương ứng với tổng số quan sát, dẫn đến kiểm định chéo loại bỏ một mẫu (`LOOCV`). Hàm `cross_validate()` tạo ra một từ điển với một số thành phần; chúng ta chỉ cần điểm số kiểm định chéo `validate()` (`MSE`) ở đây, ước tính là 24,23. `đi qua_`

Chúng ta có thể lặp lại quy trình này cho các phép khớp đa thức ngày càng phức tạp hơn. Để tự động hóa quy trình, chúng ta lại sử dụng vòng lặp `for` để lặp đi lặp lại việc khớp các hồi quy đa thức bậc từ 1 đến 5, tính toán lỗi kiểm định chéo tương ứng và lưu trữ nó trong phần tử thứ `i` của vectơ `cv_error`. Biến `d` trong vòng lặp `for` tương ứng với bậc của đa thức. Chúng ta bắt đầu bằng cách khởi tạo vectơ. Lệnh này có thể mất vài giây để chạy.

```
Trong [10]: cv_error = np.zeros(5)
H = np.array(Auto['horsepower'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1,6)):
    X = np.power.outer(H, np.arange(d+1))
    M_CV = cross_validate(M, X, Y,

                           cv=Auto.shape[0])
    cv_error[i] = np.mean(M_CV['test_score'])
lỗi cv
```

Out[10]: array([24.2315, 19.2482, 19.3350, 19.4244, 19.0332])

Như trong Hình 5.4, chúng ta thấy sự sụt giảm mạnh về `MSE` kiểm định ước tính giữa các phép khớp tuyến tính và bậc hai, nhưng sau đó không thấy sự cải thiện rõ rệt nào khi sử dụng đa thức bậc cao hơn.

Ở trên, chúng ta đã giới thiệu phương thức `outer()` của hàm `np.power()`. Phương thức `outer()` được áp dụng cho một phép toán có hai đối số, chẳng hạn như `add()`, `min()` hoặc `power()`. Nó nhận hai mảng làm đối số, sau đó tạo ra một mảng lớn hơn, trong đó phép toán được áp dụng cho từng cặp phần tử của hai mảng đó.

`.outer()`
`np.power()`

```
Trong [11]: A = np.array([3, 5, 9])
          B = np.array([2, 4])
          np.add.outer(A, B)
```

```
Out[11]: array([[ 5, 7], [ 7, 9],
               [11, 13]])
```

Trong ví dụ CV ở trên, chúng ta đã sử dụng $K = n$, nhưng tất nhiên chúng ta cũng có thể sử dụng $K < n$. Mã lệnh rất giống với ví dụ trên (và nhanh hơn đáng kể).

Ở đây, chúng ta sử dụng `KFold()` để phân chia dữ liệu thành $K = 10$ nhóm ngẫu nhiên. Chúng ta sử dụng `random_state` để thiết lập một hạt giống ngẫu nhiên và khởi tạo một vectơ `cv_error`, trong đó chúng ta sẽ lưu trữ các lỗi CV tương ứng với các phép khớp đa thức bậc từ một đến năm.

`KFold()`

```
Trong [12]: cv_error = np.zeros(5) cv =
          KFold(n_splits=10, shuffle=True,
                random_state=0) #
                sử dụng cùng một cách chia cho mỗi bậc
          for i, d in enumerate(range(1,6)):
              X = np.power.outer(H, np.arange(d+1))
              M_CV = cross_validate(M, X, Y,

                                   cv=cv)

              cv_error[i] = np.mean(M_CV['test_score'])
          lỗi cv
```

```
Out[12]: array([24.2077, 19.1853, 19.2763, 19.4785, 19.1372])
```

Lưu ý rằng thời gian tính toán ngắn hơn nhiều so với phương pháp LOOCV.

(Về nguyên tắc, thời gian tính toán cho LOOCV đối với mô hình tuyến tính bình phương tối thiểu sẽ nhanh hơn so với K-fold CV, do có sẵn công thức (5.2) cho LOOCV; tuy nhiên, hàm `cross_validate()` chung không sử dụng công thức này.) Chúng tôi vẫn thấy ít bằng chứng cho thấy việc sử dụng các số hạng đa thức bậc ba hoặc bậc cao hơn dẫn đến lỗi kiểm tra thấp hơn so với việc chỉ sử dụng phép khớp bậc hai.

Hàm `cross_validate()` rất linh hoạt và có thể nhận các cơ chế chia tách khác nhau làm đối số. Ví dụ, người ta có thể sử dụng hàm `ShuffleSplit()` để triển khai phương pháp tập dữ liệu xác thực dễ dàng như phương pháp kiểm định chéo K-fold.

Xáo trộn
Tách ra()

```
Trong [13]: validation = ShuffleSplit(n_splits=1, test_size=196,
                                   random_state=0)
                                   results =
          cross_validate(hp_model, Auto.drop(['mpg'],
                                             axis=1), Auto['mpg'], cv=validation);

          kết quả['test_score']
```

Out[13]: array([23.6166])

Có thể ước tính sự biến động của sai số kiểm tra bằng cách thực hiện các bước sau:

```
Trong [14]: validation = ShuffleSplit(n_splits=10,
                                     test_size=196,
                                     random_state=0)

results = cross_validate(hp_model,
                          Auto.drop(['mpg'], axis=1),
                          Auto['mpg'],
                          cv=validation)

results['test_score'].mean(), results['test_score'].std()
```

Out[14]: (23.8022, 1.4218)

Lưu ý rằng độ lệch chuẩn này không phải là ước tính chính xác về sự biến động lấy mẫu của điểm số trung bình bài kiểm tra hoặc điểm số cá nhân, vì các mẫu huấn luyện được chọn ngẫu nhiên chồng chéo nhau và do đó tạo ra các tương quan. Nhưng nó cho ta một ý tưởng về sự biến động Monte Carlo phát sinh do việc chọn các tập dữ liệu ngẫu nhiên khác nhau.

5.3.3 Khởi tạo (Bootstrap)

Chúng tôi minh họa việc sử dụng phương pháp bootstrap trong ví dụ đơn giản ở Mục 5.2, cũng như trong một ví dụ liên quan đến việc ước tính độ chính xác của mô hình hồi quy tuyến tính trên tập dữ liệu `Auto`.

Ước tính độ chính xác của một thống kê quan tâm

Một trong những ưu điểm lớn của phương pháp bootstrap là nó có thể được áp dụng trong hầu hết mọi tình huống. Không cần các phép tính toán học phức tạp. Mặc dù có một số cách triển khai bootstrap trong Python, nhưng việc sử dụng nó để ước tính sai số chuẩn khá đơn giản, vì vậy chúng ta sẽ viết hàm riêng của mình bên dưới cho trường hợp dữ liệu được lưu trữ trong dataframe.

Để minh họa phương pháp bootstrap, chúng ta bắt đầu với một ví dụ đơn giản. Tập dữ liệu `Portfolio` trong gói `ISLP` được mô tả trong Phần 5.2. Mục tiêu là ước tính phương sai lấy mẫu của tham số α được đưa ra trong công thức (5.7).

Chúng ta sẽ tạo một hàm `alpha_func()`, nhận đầu vào là một dataframe `D` giả định có các cột `X` và `Y`, cũng như một vectơ `idx` cho biết những quan sát nào nên được sử dụng để ước tính α . Sau đó, hàm sẽ xuất ra giá trị ước tính cho α dựa trên các quan sát đã chọn.

```
Trong [15]: Danh mục đầu tư = Load_data('Portfolio') def
alpha_func(D, idx):
    cov_ = np.cov(D[['X','Y']].loc[idx], rowvar=False) return
    ((cov_[1,1] - cov_[0,1]) /
     (cov_[0,0]+cov_[1,1]-2*cov_[0,1]))
```

Hàm này trả về ước tính cho α dựa trên việc áp dụng công thức phương sai tối thiểu (5.7) cho các quan sát được lập chỉ mục bởi đối số `idx`. Ví dụ, lệnh sau ước tính α bằng cách sử dụng tất cả 100 quan sát.

```
Trong [16]: alpha_func(Portfolio, range(100))
```

Out[16]: 0.5758

Tiếp theo, chúng ta chọn ngẫu nhiên 100 quan sát từ phạm vi (100), có thay thế. Điều này tương đương với việc xây dựng một tập dữ liệu bootstrap mới và tính toán lại α^* dựa trên tập dữ liệu mới.

```
Trong [17]: rng = np.random.default_rng(0)
alpha_func(Portfolio,
            rng.choice(100,
                       100,
                       replace=True))
```

Out[17]: 0.6074

Quá trình này có thể được khái quát hóa để tạo ra một hàm đơn giản `boot_SE()` nhằm tính toán sai số chuẩn bootstrap cho các hàm tùy ý chỉ nhận một data frame làm đối số.

```
Trong [18]: def boot_SE(func,
                      D,
                      n=None,
                      B=1000,
                      seed=0):
    rng = np.random.default_rng(seed)
    first_, second_ = 0, 0
    n = n or D.shape[0]
    for _ in range(B):
        idx = rng.choice(D.index,
                          n,
                          replace=True)
        value = func(D.loc[idx])
        first_ += value
        second_ += value**2

    return np.sqrt(second_ / B - (first_ / B)**2)
```

Hãy chú ý đến việc sử dụng `_` như một biến vòng lặp trong `for` `_` trong phạm vi (B). Điều này thường được sử dụng nếu giá trị của bộ đếm không quan trọng và chỉ đơn giản là đảm bảo vòng lặp được thực hiện B lần.

Hãy sử dụng hàm của chúng ta để đánh giá độ chính xác của ước tính α bằng cách sử dụng $B = 1.000$ lần lặp bootstrap.

```
Trong [19]: alpha_SE = boot_SE(alpha_func,
                              Danh_muc_dau_tu,
                              B=1000,
                              seed=0)

alpha_SE
```

Out[19]: 0.0912

Kết quả cuối cùng cho thấy ước tính bootstrap cho $SE(\alpha)$ là 0,0912.

Ước tính độ chính xác của mô hình hồi quy tuyến tính

Phương pháp bootstrap có thể được sử dụng để đánh giá sự biến động của các ước tính hệ số và dự đoán từ một phương pháp học thống kê. Ở đây, chúng tôi sử dụng phương pháp bootstrap để đánh giá sự biến động của...

Các ước tính cho β_0 và β_1 , hệ số chặn và hệ số góc của mô hình hồi quy tuyến tính sử dụng `mã lực` để dự đoán `mpg` trong tập dữ liệu `Auto`. Chúng tôi sẽ so sánh các ước tính thu được bằng phương pháp bootstrap với các ước tính thu được sử dụng các công thức cho $SE(\hat{\beta}_0)$ và $SE(\hat{\beta}_1)$ được mô tả trong Phần 3.1.2.

Để sử dụng hàm `boot_SE()` của chúng ta, chúng ta phải viết một hàm (tham số đầu tiên của nó) nhận một khung dữ liệu `D` và các chỉ số `idx` làm tham số duy nhất. Nhưng ở đây, chúng ta muốn sử dụng phương pháp bootstrap để xây dựng một mô hình hồi quy cụ thể, được xác định bởi một mô hình. Công thức và dữ liệu. Chúng tôi sẽ hướng dẫn cách thực hiện điều này chỉ với vài bước đơn giản.

Chúng ta bắt đầu bằng cách viết một hàm chung `boot_OLS()` để khởi tạo hệ thống. Mô hình hồi quy nhận một công thức để định nghĩa hồi quy tương ứng. Chúng ta sử dụng hàm `clone()` để tạo một bản sao của công thức có thể được áp dụng lại cho dataframe mới. Điều này có nghĩa là bất kỳ đặc trưng dẫn xuất nào như vậy như những phần được định nghĩa bởi hàm `poly()` (mà chúng ta sẽ thấy ngay sau đây), sẽ được điều chỉnh lại trên Khung dữ liệu được lấy mẫu lại.

```
Trong [20]: def boot_OLS(model_matrix, response, D, idx):
            D_ = D.loc[idx]
            Y_ = D_[phản hồi]
            X_ = clone(model_matrix).fit_transform(D_)
            return sm.OLS(Y_, X_).fit().params
```

Đây không hoàn toàn là đối số cần thiết cho hàm `boot_SE()`. Đối số đầu tiên Hai tham số xác định mô hình sẽ không thay đổi trong quá trình bootstrap. một quy trình, và chúng ta muốn đóng băng chúng. Hàm `partial()` từ mô-đun `functools` thực hiện chính xác điều này: nó nhận một hàm làm đối số, và đóng băng một số đối số của nó, bắt đầu từ bên trái. Chúng ta sử dụng nó để đóng băng hai đối số công thức mô hình đầu tiên của `boot_OLS()`.

```
Trong [21]: hp_func = partial(boot_OLS, MS(['horsepower']), 'mpg')
```

Gõ `hp_func?` sẽ cho thấy nó có hai đối số `D` và `idx` – đó là một phiên bản của `boot_OLS()` với hai đối số đầu tiên được cố định – và do đó là ideal làm đối số đầu tiên cho `boot_SE()`. Hàm `hp_func()` hiện có thể được sử dụng để tạo bootstrap. ước tính các hệ số chặn và hệ số góc bằng cách lấy mẫu ngẫu nhiên từ trong số các quan sát có hoàn trả. Trước tiên, chúng tôi chứng minh tính hữu ích của nó trên 10 mẫu bootstrap.

```
Trong [22]: rng = np.random.default_rng(0)
            np.array([hp_func(Auto,
                               rng.choice(392,
                                           392,
                                           replace=True)) cho _ trong phạm vi (10)])
```

```
Out[22]: array([[39.8806, -0.1568],
                [38.733 -0.147 ],
                [38.3173, -0.1444],
                [39.9145, -0.1578],
                [39.4335, -0.1507],
                [40.3663, -0.1591],
                [39.6233, -0.1545],
                [39.0581, -0.1495],
                [38.6669, -0.1452],
                [39.6428, -0.1556]])
```

Tiếp theo, chúng ta sử dụng hàm `boot_SE()` để tính toán sai số chuẩn của 1.000 ước tính bootstrap cho hệ số chặn và hệ số góc.

```
Trong [23]: hp_se = boot_SE(hp_func,
                          # Tự do ng,
                          B=1000,
                          seed=10)

hp_se
```

```
Out[23]: chặn mã lực      0.8488
dtype: float64      0.0074
```

Điều này cho thấy ước tính bootstrap cho $SE(\beta^0)$ là 0,85 và ước tính bootstrap cho $SE(\beta^1)$ là 0,0074. Như đã thảo luận trong Phần 3.1.2, các công thức chuẩn có thể được sử dụng để tính toán sai số chuẩn cho các hệ số hồi quy trong mô hình tuyến tính. Chúng có thể được lấy bằng cách sử dụng hàm `summarize()` từ `ISLP.sm`.

```
Trong [24]: hp_model.fit(Auto, Auto['mpg']) model_se =
summarize(hp_model.results_['std err'] model_se
```

```
Ra ngoài[24]: công suất      0.717
chặn      0.006
Tên: std err, kiểu dữ liệu: float64
```

Các ước lượng sai số chuẩn cho β^0 và β^1 thu được bằng cách sử dụng các công thức từ Mục 3.1.2 là 0,717 cho hệ số chặn và 0,006 cho hệ số góc. Điều thú vị là, những giá trị này hơi khác so với các ước lượng thu được bằng phương pháp bootstrap. Điều này có cho thấy vấn đề gì với phương pháp bootstrap không? Trên thực tế, nó cho thấy điều ngược lại. Hãy nhớ rằng các công thức chuẩn được đưa ra trong Phương trình 3.8 trên trang 75 dựa trên một số giả định nhất định. Ví dụ, chúng phụ thuộc vào tham số chưa biết σ^2 , phương sai nhiễu. Sau đó, chúng ta ước lượng σ^2 bằng cách sử dụng RSS. Mặc dù công thức tính sai số chuẩn không phụ thuộc vào việc mô hình tuyến tính là chính xác, nhưng ước lượng cho σ^2 thì có. Chúng ta thấy trong Hình 3.8 trên trang 99 rằng có mối quan hệ phi tuyến tính trong dữ liệu, và do đó phần dư từ phép hồi quy tuyến tính sẽ bị thổi phồng, và σ^2 cũng vậy. Thứ hai, các công thức chuẩn giả định (một cách không thực tế) rằng các x_i là cố định, và tất cả sự biến thiên đến từ sự biến thiên trong các sai số. Phương pháp bootstrap không dựa trên bất kỳ giả định nào trong số này, và do đó, nó có khả năng đưa ra ước tính chính xác hơn về sai số chuẩn của β^0 và β^1 so với kết quả từ `sm.OLS`.

Dưới đây, chúng tôi tính toán các ước lượng sai số chuẩn bootstrap và các ước lượng hồi quy tuyến tính chuẩn thu được từ việc khớp mô hình bậc hai với dữ liệu. Vì mô hình này cung cấp sự phù hợp tốt với dữ liệu (Hình 3.8), nên hiện nay có sự tương ứng tốt hơn giữa các ước lượng bootstrap và các ước lượng chuẩn của $SE(\beta^0)$, $SE(\beta^1)$ và $SE(\beta^2)$.

```
Trong [25]: quad_model = MS([poly('horsepower', 2, raw=True)]) quad_func =
partial(boot_OLS, quad_model, 'mpg')

boot_SE(quad_func,
Auto,
B=1000)
```

```
Out[25]: intercept                2.067840
         poly(horsepower, 2, raw=True)[0]    0.033019
         poly(horsepower, 2, raw=True)[1] dtype: float64    0.000120
```

Chúng tôi so sánh kết quả với sai số chuẩn được tính toán bằng `sm.OLS()`.

```
Trong [26]: M = sm.OLS(Auto['mpg'],
                      quad_model.fit_transform(Auto))
           summarize(M.fit())['std err']
```

```
Out[26]: intercept                1.800
         poly(horsepower, 2, raw=True)[0]    0.031
         poly(horsepower, 2, raw=True)[1]    0.000
         Tên: std err, kiểu dữ liệu: float64
```

5.4 Bài tập

Khái niệm

- Sử dụng các tính chất thống kê cơ bản của phương sai, cũng như phép tính vi phân biến đơn, hãy suy ra (5.6). Nói cách khác, hãy chứng minh rằng α được cho bởi (5.6) thực sự tối thiểu hóa $\text{Var}(\alpha X + (1 - \alpha)Y)$.
- Bây giờ chúng ta sẽ tính xác suất để một quan sát nhất định là một phần của của một mẫu bootstrap. Giả sử chúng ta thu được một mẫu bootstrap từ một tập hợp gồm n quan sát.
 - Xác suất để quan sát bootstrap đầu tiên là bao nhiêu?
Không phải là quan sát thứ j từ mẫu ban đầu? Hãy giải thích điều đó. trả lời.
 - Xác suất để quan sát bootstrap thứ hai là bao nhiêu?
Liệu quan sát thứ j có phải là từ mẫu ban đầu không?
 - Hãy lập luận rằng xác suất để quan sát thứ j không nằm trong Mẫu bootstrap là $(1 - 1/n)^n$.
 - Khi $n = 5$, xác suất để quan sát thứ j là bao nhiêu?
trong mẫu bootstrap?
 - Khi $n = 100$, xác suất để quan sát thứ j là bao nhiêu?
Nó có nằm trong mẫu bootstrap không?
 - Khi $n = 10.000$, xác suất để quan sát thứ j là bao nhiêu?
Câu hỏi nằm trong mẫu bootstrap phải không?
 - Tạo một đồ thị hiển thị, với mỗi giá trị nguyên của n từ 1 đến 100.000, xác suất để quan sát thứ j nằm trong phạm vi đó.
Ví dụ bootstrap. Hãy bình luận về những gì bạn quan sát được.
 - Bây giờ chúng ta sẽ khảo sát bằng số xác suất để một mẫu bootstrap có kích thước $n = 100$ chứa quan sát thứ j . Ở đây
 $j = 4$. Đầu tiên, chúng ta tạo một mảng lưu trữ các giá trị sẽ được ghi đè sau đó bằng hàm `np.empty()`. Sau đó, chúng ta sử dụng `np.empty()`

Chúng ta liên tục tạo các mẫu bootstrap, và mỗi lần chúng ta ghi lại xem quan sát thứ năm có nằm trong mẫu bootstrap đó hay không.

```
rng = np.random.default_rng(10)
store = np.empty(10000)
for i in range(10000):
    store[i] = np.sum(rng.choice(100, replace=True) == 4)
    >0
np.mean(store)
```

Nhận xét về kết quả thu được.

3. Bây giờ chúng ta sẽ xem xét lại phương pháp kiểm định chéo k-fold.

(a) Giải thích cách thức triển khai phương pháp kiểm định chéo k-fold. (b)

Ưu điểm và nhược điểm của phương pháp kiểm định chéo k-fold là gì?

Xác thực liên quan đến:

i. Phương pháp tập dữ liệu kiểm định? ii.

Phương pháp LOOCV?

4. Giả sử chúng ta sử dụng một phương pháp học thống kê nào đó để dự đoán giá trị phản hồi Y cho một giá trị cụ thể của biến dự báo X .

Hãy mô tả cẩn thận cách chúng ta có thể ước tính độ lệch chuẩn của dự đoán.

Đã áp dụng

5. Trong Chương 4, chúng ta đã sử dụng hồi quy logistic để dự đoán xác suất **vỡ nợ** dựa trên **thu nhập** và **số dư** trên tập dữ liệu **Vỡ nợ**. Bây giờ chúng ta sẽ ước tính sai số kiểm định của mô hình hồi quy logistic này bằng cách sử dụng phương pháp tập dữ liệu xác thực. Đừng quên thiết lập một hạt giống ngẫu nhiên trước khi bắt đầu phân tích.

(a) Xây dựng mô hình hồi quy logistic sử dụng **thu nhập** và **số dư** để dự đoán **khả năng vỡ nợ**.

(b) Sử dụng phương pháp tập dữ liệu kiểm định, hãy ước tính sai số kiểm định của mô hình này. Để làm được điều này, bạn phải thực hiện các bước sau: i. Chia tập dữ liệu mẫu

thành tập huấn luyện và tập kiểm định. ii. Xây dựng mô hình hồi quy logistic đa biến chỉ sử dụng các quan sát trong tập huấn luyện. iii. Đưa ra dự đoán về tình trạng vỡ nợ cho mỗi

cá nhân trong tập kiểm định bằng cách tính xác suất hậu nghiệm về tình trạng vỡ nợ của cá nhân đó, và phân loại cá nhân đó vào nhóm **vỡ nợ** nếu xác suất hậu nghiệm lớn hơn 0,5.

iv. Tính toán sai số của tập dữ liệu kiểm định, tức là tỷ lệ các quan sát trong tập dữ liệu kiểm định bị phân loại sai.

(c) Lặp lại quy trình trong (b) ba lần, sử dụng ba cách chia khác nhau các quan sát thành tập huấn luyện và tập kiểm chứng. Nhận xét về kết quả thu được.

(d) Bây giờ hãy xem xét một mô hình hồi quy logistic dự đoán xác suất **vỡ nợ** bằng cách sử dụng **thu nhập**, **số dư** và một biến giả cho **sinh viên**. Ước tính sai số kiểm định cho mô hình này bằng cách sử dụng phương pháp tập dữ liệu kiểm định. Nhận xét về việc có nên đưa biến giả cho **sinh viên** vào hay không.

6. Chúng tôi tiếp tục xem xét việc sử dụng mô hình hồi quy logistic để dự đoán xác suất **vỡ nợ** bằng cách sử dụng **thu nhập** và **số dư** trên tập dữ liệu **Vỡ nợ**. Cụ thể, bây giờ chúng tôi sẽ tính toán ước lượng sai số chuẩn của các hệ số hồi quy logistic **thu nhập** và **số dư** theo hai cách khác nhau: (1) sử dụng phương pháp bootstrap, và (2) sử dụng công thức chuẩn để tính toán sai số chuẩn trong hàm **sm.GLM()**. Đừng quên đặt một hạt giống ngẫu nhiên trước khi bắt đầu phân tích của bạn.

(a) Sử dụng các hàm **summarize()** và **sm.GLM()**, hãy xác định sai số chuẩn ước tính cho các hệ số liên quan đến **thu nhập** và **số dư** trong mô hình hồi quy logistic đa biến sử dụng cả hai biến dự báo này.

(b) Viết một hàm, **boot_fn()**, nhận đầu vào là tập dữ liệu **Mặc định** cũng như chỉ số của các quan sát, và xuất ra các ước tính hệ số cho **thu nhập** và **số dư** trong mô hình hồi quy logistic đa biến. (c) Theo ví dụ bootstrap trong bài thực hành, sử dụng hàm **boot_fn()** của bạn để ước tính

sai số chuẩn của các hệ số hồi quy logistic cho **thu nhập** và **số dư**.

(d) Nhận xét về các sai số chuẩn ước tính thu được bằng cách sử dụng hàm **sm.GLM()** và sử dụng phương pháp bootstrap.

7. Trong Mục 5.1.2 và 5.1.3, chúng ta đã thấy rằng hàm **cross_validate()** có thể được sử dụng để tính toán ước lượng lỗi kiểm định LOOCV.

Ngoài ra, người ta cũng có thể tính toán các đại lượng đó chỉ bằng cách sử dụng **sm.GLM()** và phương thức **predict()** của mô hình đã được hiệu chỉnh trong một vòng lặp for. Bây giờ bạn sẽ sử dụng cách tiếp cận này để tính toán lỗi LOOCV cho một mô hình hồi quy logistic đơn giản trên tập dữ liệu **Weekly**. Nhớ lại rằng trong bối cảnh các bài toán phân loại, lỗi LOOCV được đưa ra trong (5.4).

(a) Xây dựng mô hình hồi quy logistic dự đoán **Hướng** bằng cách sử dụng **Độ trễ 1** và **Lag2**.

(b) Xây dựng mô hình hồi quy logistic dự đoán **Hướng** bằng cách sử dụng **Độ trễ 1** và **Lag2** sử dụng tất cả các quan sát ngoại trừ quan sát đầu tiên.

(c) Sử dụng mô hình từ (b) để dự đoán hướng của quan sát đầu tiên. Bạn có thể làm điều này bằng cách dự đoán rằng quan sát đầu tiên sẽ đi lên nếu $P(\text{Hướng} = \text{"Đi lên"} | \text{Độ trễ 1}, \text{Độ trễ 2}) > 0,5$. Quan sát này đã được phân loại chính xác chưa?

(d) Viết một vòng lặp for từ $i = 1$ đến $i = n$, trong đó n là số lượng quan sát trong tập dữ liệu, thực hiện từng bước sau:

- i. Xây dựng mô hình hồi quy logistic sử dụng tất cả các quan sát ngoại trừ quan sát thứ i để dự đoán hướng đi bằng cách sử dụng độ trễ 1 và độ trễ 2.
- ii. Tính xác suất hậu nghiệm của việc thị trường tăng giá đối với quan sát thứ i .
- iii. Sử dụng xác suất hậu nghiệm cho quan sát thứ i để dự đoán liệu thị trường có tăng hay không.
- iv. Xác định xem có sai sót nào xảy ra trong việc dự đoán hướng của quan sát thứ i hay không. Nếu có sai sót, hãy ghi là 1, ngược lại ghi là 0.

(e) Lấy trung bình cộng của n số thu được ở (d) để có được ước lượng LOOCV cho sai số kiểm định. Nhận xét về kết quả.

8. Bây giờ chúng ta sẽ tiến hành kiểm định chéo trên một tập dữ liệu mô phỏng.

(a) Tạo bộ dữ liệu mô phỏng như sau:

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = x - 2 * x**2 + rng.normal(size=100)
```

Trong tập dữ liệu này, n và p bằng bao nhiêu? Hãy viết mô hình được sử dụng để tạo ra dữ liệu dưới dạng phương trình.

(b) Tạo biểu đồ phân tán của X so với Y . Nhận xét về những gì bạn tìm thấy. (c) Đặt một hạt giống ngẫu nhiên, sau đó tính toán các lỗi LOOCV thu được từ việc khớp bốn mô hình sau bằng phương pháp bình phương tối thiểu:

$$\text{Tôi. } Y = \beta_0 + \beta_1 X + \text{ii.}$$

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \text{iii. } Y = \beta_0$$

$$+ \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \text{iv. } Y = \beta_0 + \beta_1 X + \beta_2 X^2$$

$$+ \beta_3 X^3 + \beta_4 X^4 + .$$

Lưu ý rằng bạn có thể thấy hữu ích khi sử dụng hàm `data.frame()` để tạo một tập dữ liệu duy nhất chứa cả X và Y . (d) Lặp lại (c) bằng cách sử

dụng một hạt giống ngẫu nhiên khác và báo cáo kết quả của bạn.

Kết quả của bạn có giống với kết quả bạn nhận được ở phần (c) không? Tại sao?

(e) Mô hình nào trong (c) có lỗi LOOCV nhỏ nhất? Đây có phải là điều bạn mong đợi không?

Giải thích câu trả lời của bạn. (f) Nhận xét về ý nghĩa

thống kê của các ước lượng hệ số thu được từ việc khớp từng mô hình trong (c) bằng phương pháp bình phương nhỏ nhất. Các kết quả này có phù hợp với các kết luận rút ra dựa trên kết quả kiểm định chéo không?

9. Bây giờ chúng ta sẽ xem xét bộ dữ liệu nhà ở ở Boston, từ ISLP. thư viện.

(a) Dựa trên tập dữ liệu này, hãy đưa ra ước tính về dân số.

Giá trị trung bình của `medv`. Gọi giá trị ước tính này là μ^* .

5. Phương pháp lấy mẫu lại

- (b) Cung cấp ước tính về sai số chuẩn của μ^* . Giải thích điều này. kết quả.

Gợi ý: Chúng ta có thể tính sai số chuẩn của trung bình mẫu bằng cách...
chia độ lệch chuẩn mẫu cho căn bậc hai của
số lượng quan sát.

- (c) Bây giờ hãy ước tính sai số chuẩn của μ^* bằng phương pháp bootstrap. Làm thế nào
Điều này có tương đồng với câu trả lời của bạn ở mục (b) không?

- (d) Dựa trên ước tính bootstrap của bạn từ (c), hãy cung cấp khoảng tin cậy 95%
cho giá trị trung bình của `medv`. So sánh nó với kết quả.
thu được bằng cách sử dụng `Boston['medv'].std()` và hai tiêu chuẩn
quy tắc lỗi (3.9).

Gợi ý: Bạn có thể ước tính khoảng tin cậy 95% bằng cách sử dụng...
công thức $[\mu^* - 2SE(\mu^*), \mu^* + 2SE(\mu^*)]$.

- (e) Dựa trên tập dữ liệu này, hãy đưa ra ước tính, μ^{med} , cho giá trị trung vị
Giá trị của `medv` trong dân số.

- (f) Bây giờ chúng ta muốn ước tính sai số chuẩn của μ^{med} . Thật không may, không
có công thức đơn giản nào để tính toán sai số chuẩn.
sai số của trung vị. Thay vào đó, hãy ước tính sai số chuẩn của
Trung vị được tính bằng phương pháp bootstrap. Hãy bình luận về kết quả của bạn.

- (g) Dựa trên tập dữ liệu này, hãy ước tính phân vị thứ mười của `medv` trong các khu
vực điều tra dân số Boston. Gọi giá trị này là $\mu^{0.1}$.
(Bạn có thể sử dụng hàm `np.percentile()` .)

`np.`
`percentile()`

- (h) Sử dụng phương pháp bootstrap để ước tính sai số chuẩn của $\mu^{0.1}$. Nhận xét về
kết quả của bạn.

6



Lựa chọn và điều chỉnh mô hình tuyến tính

Trong thiết lập hồi quy, mô hình tuyến tính chuẩn được sử dụng.

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \quad (6.1)$$

Mô hình này thường được sử dụng để mô tả mối quan hệ giữa biến phản hồi Y và một tập hợp các biến $X_1, X_2, \dots, X_p$. Chúng ta đã thấy trong Chương 3 rằng người ta thường sử dụng phương pháp bình phương tối thiểu để khớp mô hình này.

Trong các chương tiếp theo, chúng ta sẽ xem xét một số phương pháp mở rộng khung mô hình tuyến tính. Trong Chương 7, chúng ta sẽ khái quát hóa (6.1) để phù hợp với các mối quan hệ phi tuyến tính, nhưng vẫn có tính chất cộng, trong khi ở Chương 8 và 10, chúng ta sẽ xem xét các mô hình phi tuyến tính tổng quát hơn nữa. Tuy nhiên, mô hình tuyến tính có những ưu điểm riêng biệt về mặt suy luận và, đối với các vấn đề thực tế, thường cạnh tranh đáng ngạc nhiên so với các phương pháp phi tuyến tính. Do đó, trước khi chuyển sang thể giới phi tuyến tính, trong chương này, chúng ta sẽ thảo luận một số cách để cải thiện mô hình tuyến tính đơn giản bằng cách thay thế phương pháp bình phương tối thiểu thông thường bằng một số quy trình khớp dữ liệu thay thế khác.

Tại sao chúng ta lại muốn sử dụng một phương pháp hiệu chỉnh khác thay vì phương pháp bình phương tối thiểu? Như chúng ta sẽ thấy, các phương pháp hiệu chỉnh thay thế có thể mang lại độ chính xác dự đoán tốt hơn và khả năng giải thích mô hình cao hơn.

- Độ chính xác dự đoán: Giả sử mối quan hệ thực sự giữa biến phản hồi và các biến dự đoán là tuyến tính, thì các ước lượng bình phương nhỏ nhất sẽ có độ sai lệch thấp. Nếu n —tức là nếu n , số lượng quan sát, lớn hơn nhiều so với p , số lượng biến—thì các ước lượng bình phương nhỏ nhất cũng có xu hướng có phương sai thấp, và do đó sẽ hoạt động tốt trên các quan sát thử nghiệm. Tuy nhiên, nếu n không lớn hơn nhiều so với p , thì có thể có rất nhiều biến thiên trong sự phù hợp của bình phương nhỏ nhất, dẫn đến hiện tượng quá khớp và do đó dự đoán kém trên các quan sát trong tương lai không được sử dụng trong quá trình huấn luyện mô hình. Và nếu $p > n$, thì không còn ước lượng hệ số bình phương nhỏ nhất duy nhất nữa: có vô số hệ số khác nhau.

Nhiều giải pháp. Mỗi giải pháp bình phương tối thiểu này đều cho sai số bằng 0 trên dữ liệu huấn luyện, nhưng thường cho hiệu suất rất kém trên tập dữ liệu kiểm tra do phương sai cực kỳ cao.¹ Bằng cách ràng buộc hoặc thu nhỏ các hệ số ước tính, chúng ta thường có thể giảm đáng kể phương sai với chi phí là sự gia tăng không đáng kể về độ lệch. Điều này có thể dẫn đến những cải thiện đáng kể về độ chính xác mà chúng ta có thể dự đoán phản hồi cho các quan sát không được sử dụng trong quá trình huấn luyện mô hình.

- Khả năng giải thích mô hình: Thông thường, một số hoặc nhiều biến được sử dụng trong mô hình hồi quy đa biến thực tế không liên quan đến biến phản hồi. Việc đưa các biến không liên quan như vậy vào dẫn đến sự phức tạp không cần thiết trong mô hình kết quả. Bằng cách loại bỏ các biến này—tức là bằng cách đặt các ước lượng hệ số tương ứng bằng không—chúng ta có thể thu được một mô hình dễ giải thích hơn. Tuy nhiên, phương pháp bình phương tối thiểu rất khó có thể cho ra bất kỳ ước lượng hệ số nào chính xác bằng không. Trong chương này, chúng ta sẽ xem xét một số phương pháp để tự động thực hiện lựa chọn đặc trưng hoặc lựa chọn biến—tức là để loại bỏ các biến không liên quan khỏi mô hình hồi quy đa biến.

lựa chọn
tính năng
lựa chọn
biến

Có rất nhiều phương án thay thế, cả cổ điển và hiện đại, cho việc sử dụng phương pháp bình phương nhỏ nhất để phù hợp (6.1). Trong chương này, chúng ta sẽ thảo luận về ba lớp phương pháp quan trọng.

- Lựa chọn tập con. Phương pháp này bao gồm việc xác định một tập con của p biến dự báo mà chúng ta tin là có liên quan đến biến phản hồi. Sau đó, chúng ta sử dụng phương pháp bình phương nhỏ nhất để xây dựng mô hình trên tập hợp các biến đã được thu hẹp.
- Phương pháp co rút (Shrinkage). Phương pháp này bao gồm việc xây dựng một mô hình với tất cả p biến dự báo. Tuy nhiên, các hệ số ước lượng được thu hẹp về phía 0 so với các ước lượng bình phương nhỏ nhất. Sự co rút này (còn được gọi là chuẩn hóa) có tác dụng giảm phương sai. Tùy thuộc vào loại co rút được thực hiện, một số hệ số có thể được ước lượng chính xác bằng 0. Do đó, các phương pháp co rút cũng có thể thực hiện chọn lọc biến.
- Giảm chiều dữ liệu. Phương pháp này bao gồm việc chiếu p biến dự báo vào một không gian con M chiều, trong đó $M < p$. Điều này được thực hiện bằng cách tính toán M tổ hợp tuyến tính khác nhau, hay phép chiếu, của các biến. Sau đó, M phép chiếu này được sử dụng làm biến dự báo để phù hợp với mô hình hồi quy tuyến tính bằng phương pháp bình phương tối thiểu.

Trong các phần tiếp theo, chúng tôi sẽ mô tả chi tiết hơn từng phương pháp này, cùng với những ưu điểm và nhược điểm của chúng. Mặc dù chương này mô tả các phần mở rộng và sửa đổi đối với mô hình tuyến tính cho hồi quy được trình bày trong Chương 3, nhưng các khái niệm tương tự cũng áp dụng cho các phương pháp khác, chẳng hạn như các mô hình phân loại được trình bày trong Chương 4.

¹Khi p_n , giải pháp bình phương tối thiểu có tổng bình phương hệ số nhỏ nhất đôi khi có thể hoạt động khá tốt. Xem Mục 10.8 để thảo luận chi tiết hơn.

6.1 Lựa chọn tập con

Trong phần này, chúng ta sẽ xem xét một số phương pháp lựa chọn tập con của các biến dự báo. Các phương pháp này bao gồm lựa chọn tập con tốt nhất và quy trình lựa chọn mô hình từng bước.

6.1.1 Lựa chọn tập con tốt nhất

Để thực hiện lựa chọn tập con tốt nhất, chúng ta tiến hành hồi quy bình phương tối thiểu riêng biệt cho mỗi tổ hợp có thể có của p biến dự đoán. Nghĩa là, chúng ta tiến hành tất cả p mô hình chứa chính xác một biến dự đoán, tất cả $p = p(p-1)/2$ mô hình chứa chính xác hai biến dự đoán, v.v. Sau đó, chúng ta xem xét tất cả các mô hình thu được, với mục tiêu xác định mô hình tốt nhất.

lựa chọn tập
con tốt nhất

Bài toán chọn mô hình tốt nhất trong số 2^p khả năng được xem xét bởi phương pháp chọn tập con tốt nhất không phải là đơn giản. Bài toán này thường được chia thành hai giai đoạn, như mô tả trong Thuật toán 6.1.

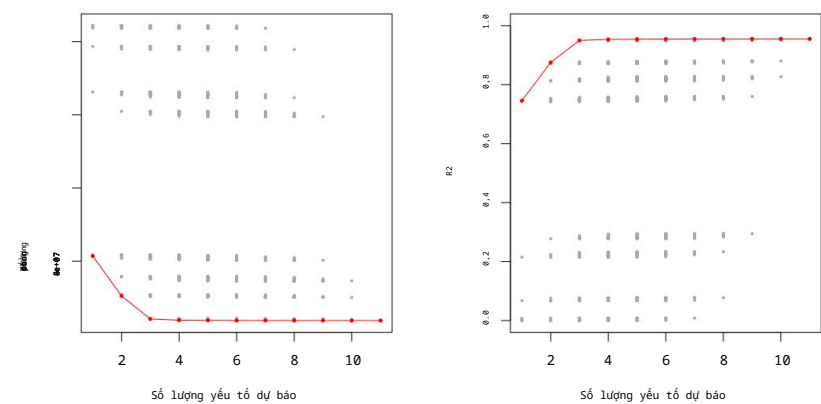
Thuật toán 6.1 Lựa chọn tập con tốt nhất

- Gọi M_0 là mô hình trống, không chứa biến dự đoán nào. Mô hình này chỉ đơn giản là dự đoán giá trị trung bình mẫu cho mỗi quan sát.
- Với $k = 1, 2, \dots, p$:
 - Phù hợp với tất cả k các mô hình chứa chính xác k biến dự đoán.
các giá trị p (b) Chọn giá trị tốt nhất trong số các giá trị p này các mô hình, và gọi nó là M_k . Ở đây, "tốt nhất" được định nghĩa là có RSS nhỏ nhất, hoặc tương đương với R^2 lớn nhất.
- Chọn một mô hình tốt nhất trong số $M_0, \dots, M_p$ bằng cách sử dụng sai số dự đoán trên tập dữ liệu xác thực, C_p (AIC), BIC hoặc R^2 hiệu chỉnh.
Hoặc sử dụng phương pháp kiểm định chéo.

Trong Thuật toán 6.1, Bước 2 xác định mô hình tốt nhất (trên dữ liệu huấn luyện) cho mỗi kích thước tập con, nhằm giảm bài toán từ một trong 2^p mô hình khả thi xuống một trong $p+1$ mô hình khả thi. Trong Hình 6.1, các mô hình này tạo thành đường biên dưới được thể hiện bằng màu đỏ.

Để chọn ra một mô hình tốt nhất, chúng ta chỉ cần lựa chọn trong số $p+1$ tùy chọn này. Việc này cần được thực hiện cẩn thận, bởi vì hệ số tương quan bình phương (RSS) của $p+1$ mô hình này giảm dần và hệ số tương quan bình phương (R^2) tăng dần khi số lượng đặc trưng được đưa vào mô hình tăng lên.

Do đó, nếu chúng ta sử dụng các số liệu thống kê này để chọn mô hình tốt nhất, thì cuối cùng chúng ta sẽ luôn có một mô hình bao gồm tất cả các biến. Vấn đề là RSS thấp hoặc R^2 cao cho thấy một mô hình có lỗi huấn luyện thấp, trong khi chúng ta muốn chọn một mô hình có lỗi kiểm thử thấp. (Như đã trình bày trong Chương 2 ở Hình 2.9-2.11, lỗi huấn luyện thường nhỏ hơn đáng kể so với lỗi kiểm thử, và lỗi huấn luyện thấp không có nghĩa là lỗi kiểm thử thấp.) Do đó, ở Bước 3, chúng ta sử dụng lỗi trên tập dữ liệu xác thực, C_p , BIC hoặc R^2 hiệu chỉnh để lựa chọn giữa $M_0, M_1, \dots, M_p$. Nếu sử dụng phương pháp kiểm định chéo để chọn mô hình tốt nhất, thì Bước 2 được lặp lại trên mỗi fold huấn luyện, và các lỗi xác thực được tính trung bình để chọn giá trị k tốt nhất.



HÌNH 6.1. Đối với mỗi mô hình khả thi chứa một tập hợp con của mười biến dự đoán. Trong bộ dữ liệu *Tín dụng*, RSS và R2 được hiển thị. Đường viền màu đỏ theo dõi... Mô hình tốt nhất cho một số lượng biến dự đoán nhất định, theo RSS và R2. Tuy nhiên, Tập dữ liệu chỉ chứa mười biến dự đoán, trục x có phạm vi từ 1 đến 11, vì một trong số các biến là biến phân loại và nhận ba giá trị, dẫn đến việc tạo ra hai biến giả.

Sau đó, mô hình M_k phù hợp với toàn bộ tập dữ liệu huấn luyện sẽ được cung cấp cho người được chọn. k. Các phương pháp này được thảo luận trong Phần 6.1.3.

Một ứng dụng của phương pháp chọn tập con tốt nhất được minh họa trong Hình 6.1. Mỗi điểm được vẽ trên đồ thị tương ứng với mô hình hồi quy bình phương nhỏ nhất được hiệu chỉnh bằng cách sử dụng... một tập hợp con khác của 10 yếu tố dự báo trong bộ dữ liệu *Tín dụng*, được thảo luận trong Chương 3. Ở đây, *vùng* biến đổi là một biến định tính ba cấp độ.

và do đó được biểu diễn bằng hai biến giả, được chọn riêng biệt trong trường hợp này. Vì vậy, có tổng cộng 11 biến khả dĩ, có thể được đưa vào mô hình. Chúng tôi đã vẽ biểu đồ thống kê RSS và R2. Đối với mỗi mô hình, tùy thuộc vào số lượng biến. Các đường cong màu đỏ Kết nối các mô hình tốt nhất cho từng kích thước mô hình, theo RSS hoặc R2. Hình vẽ cho thấy, đúng như dự đoán, các đại lượng này sẽ cải thiện khi số lượng tăng lên. số lượng biến tăng lên; tuy nhiên, từ mô hình ba biến trở đi, sự khác biệt là rất nhỏ. Sự cải thiện về RSS và R2 là kết quả của việc bổ sung thêm các biến dự đoán.

Mặc dù chúng tôi đã trình bày phương pháp chọn tập con tốt nhất ở đây cho phương pháp bình phương tối thiểu. Đối với hồi quy logistic, các ý tưởng tương tự cũng áp dụng cho các loại mô hình khác, chẳng hạn như mô hình logistic. hồi quy. Trong trường hợp hồi quy logistic, thay vì sắp xếp các mô hình theo Thay vì sử dụng RSS trong Bước 2 của Thuật toán 6.1, chúng ta sử dụng độ lệch, một thước đo đóng vai trò tương tự như RSS cho một lớp mô hình rộng hơn. Độ lệch là âm hai lần logarit xác suất tối đa; độ lệch càng nhỏ, Càng vừa vặn càng tốt.

sự lệch lạc

Mặc dù lựa chọn tập con tốt nhất là một phương pháp đơn giản và hấp dẫn về mặt khái niệm, nhưng nó lại gặp phải những hạn chế về mặt tính toán. Số lượng các tập con có thể có Các mô hình cần được xem xét tăng nhanh khi p tăng. Nói chung, Có 2^p mô hình liên quan đến các tập con của p biến dự đoán. Vì vậy, nếu $p = 10$, Khi đó, có khoảng 1.000 mô hình khả thi cần được xem xét, và nếu Nếu $p = 20$, thì sẽ có hơn một triệu khả năng! Do đó, việc lựa chọn tập con tốt nhất trở nên không khả thi về mặt tính toán đối với các giá trị của p lớn hơn

Thuật toán 6.2 Lựa chọn từng bước tiến lên

1. Gọi M_0 là mô hình trống, không chứa biến dự đoán nào.
2. Đối với $k = 0, \dots, p - 1$:

(a) Hãy xem xét tất cả $p - k$ mô hình bổ sung thêm một biến dự đoán vào các biến dự đoán trong M_k . (b) Chọn mô hình tốt nhất trong số $p - k$ mô hình này và gọi nó là M_{k+1} . Ở đây, "tốt nhất" được định nghĩa là có RSS nhỏ nhất hoặc R^2 cao nhất.
3. Chọn một mô hình tốt nhất trong số $M_0, \dots, M_p$ bằng cách sử dụng sai số dự đoán trên tập dữ liệu kiểm định, C_p (AIC), BIC hoặc R^2 hiệu chỉnh. Hoặc sử dụng phương pháp kiểm định chéo.

Khoảng 40, ngay cả với những máy tính hiện đại cực nhanh. Có những phương pháp rút gọn tính toán –còn gọi là kỹ thuật nhánh và cận–để loại bỏ một số lựa chọn, nhưng những phương pháp này có những hạn chế khi p lớn. Chúng cũng chỉ hoạt động đối với hồi quy tuyến tính bình phương tối thiểu. Tiếp theo, chúng tôi sẽ trình bày các phương pháp thay thế hiệu quả về mặt tính toán cho việc lựa chọn tập con tốt nhất.

6.1.2 Lựa chọn từng bước

Vì lý do tính toán, việc lựa chọn tập con tốt nhất không thể áp dụng với p rất lớn. Việc lựa chọn tập con tốt nhất cũng có thể gặp phải các vấn đề thống kê khi p lớn. Không gian tìm kiếm càng lớn, khả năng tìm thấy các mô hình trông tốt trên dữ liệu huấn luyện càng cao, ngay cả khi chúng có thể không có khả năng dự đoán trên dữ liệu tương lai. Do đó, không gian tìm kiếm khổng lồ có thể dẫn đến hiện tượng quá khớp và phương sai cao của các ước tính hệ số.

Vì cả hai lý do trên, các phương pháp từng bước, vốn khám phá một tập hợp mô hình hạn chế hơn nhiều, là những lựa chọn thay thế hấp dẫn cho phương pháp chọn tập con tốt nhất.

Lựa chọn từng bước tiến lên

Phương pháp lựa chọn từng bước tiến (forward stepwise selection) là một giải pháp thay thế hiệu quả về mặt tính toán so với phương pháp lựa chọn tập con tốt nhất (best subset selection). Trong khi phương pháp lựa chọn tập con tốt nhất xem xét tất cả 2^p mô hình có thể chứa các tập con của p biến dự đoán, phương pháp lựa chọn từng bước tiến chỉ xem xét một tập hợp mô hình nhỏ hơn nhiều. Phương pháp lựa chọn từng bước tiến bắt đầu với một mô hình không chứa biến dự đoán nào, sau đó thêm các biến dự đoán vào mô hình, từng biến một, cho đến khi tất cả các biến dự đoán đều có trong mô hình. Cụ thể, ở mỗi bước, biến số mang lại sự cải thiện bổ sung lớn nhất cho độ phù hợp sẽ được thêm vào mô hình. Nói một cách chính xác hơn, quy trình lựa chọn từng bước tiến lên được trình bày trong Thuật toán 6.2.

lựa chọn
từng bước
tiến lên

Không giống như phương pháp chọn tập con tốt nhất, vốn liên quan đến việc khớp 2^p mô hình, phương pháp chọn từng bước tiến lên liên quan đến việc khớp một mô hình trống, cùng với $p - k$ mô hình trong lần lặp thứ k , với $k = 0, \dots, p - 1$. Điều này tương đương với tổng cộng $1 + p(p - 1)/2$ mô hình. Đây là một sự khác biệt đáng kể: khi

# Biến Tập con tốt nhất	Tiến lên từng bước
Một đánh giá	
Xếp hạng hai , xếp hạng thu nhập, thu nhập	
Ba mức xếp hạng, thu nhập, xếp hạng sinh viên, thu nhập, sinh viên	
Bốn lá bài, xếp hạng thu nhập, thu nhập,	
	sinh viên, giới hạn sinh viên, giới hạn

BẢNG 6.1. Bốn mô hình được chọn đầu tiên để lựa chọn tập con tốt nhất và chuyển tiếp Lựa chọn từng bước trên tập dữ liệu *Tin dụng* . Ba mô hình đầu tiên giống hệt nhau nhưng Mô hình thứ tư có sự khác biệt.

$p = 20$, việc lựa chọn tập con tốt nhất đòi hỏi phải phù hợp với 1.048.576 mô hình, trong khi đó Phương pháp lựa chọn từng bước tiến lên chỉ yêu cầu lắp đặt 211 mô hình.2

Trong Bước 2(b) của Thuật toán 6.2, chúng ta phải xác định mô hình tốt nhất từ trong số các p k bổ sung thêm một biến dự đoán cho M_k . Chúng ta có thể Bạn có thể thực hiện điều này bằng cách đơn giản chọn mô hình có RSS thấp nhất hoặc cao nhất. R2. Tuy nhiên, ở Bước 3, chúng ta phải xác định mô hình tốt nhất trong một tập hợp các mô hình. các mô hình với số lượng biến khác nhau. Điều này khó khăn hơn và được thảo luận trong Mục 6.1.3.

Ưu điểm về mặt tính toán của phương pháp chọn lọc từng bước tiến lên so với phương pháp chọn tập con tốt nhất. Việc lựa chọn rất rõ ràng. Mặc dù phương pháp chọn từng bước tiến thường cho kết quả tốt trong thực tế, Không có gì đảm bảo sẽ tìm được mô hình tốt nhất trong số tất cả 2^p mô hình chứa các tập con của p biến dự đoán. Ví dụ, giả sử trong một Với tập dữ liệu có $p = 3$ biến dự đoán, mô hình một biến tốt nhất có thể là gì? chứa X_1 , và mô hình hai biến tốt nhất có thể thay vào đó chứa X_2 . và X_3 . Khi đó, phương pháp chọn lọc từng bước tiến lên sẽ không thể chọn được kết quả tốt nhất có thể. mô hình hai biến, vì M_1 sẽ chứa X_1 , nên M_2 cũng phải chứa X_1 cùng với một biến bổ sung khác.

Bảng 6.1 hiển thị bốn mô hình được chọn đầu tiên cho tập con tốt nhất. và phương pháp chọn lọc từng bước tiến lên trên tập dữ liệu *Tin dụng* minh họa hiện tượng này. Cả phương pháp chọn tập con tốt nhất và phương pháp chọn lọc từng bước tiến lên đều chọn *xếp hạng* cho mô hình một biến tốt nhất, sau đó bao gồm *thu nhập* và *sinh viên*. đối với các mô hình hai và ba biến. Tuy nhiên, việc lựa chọn tập con tốt nhất thay thế việc xếp hạng *bằng* thế *trong* mô hình bốn biến, trong khi phương pháp từng bước tiến lên Việc lựa chọn phải duy trì *xếp hạng* trong mô hình bốn biến của nó. Trong ví dụ này, Hình 6.1 cho thấy không có nhiều khác biệt giữa mô hình ba biến và bốn biến về RSS, vì vậy cả hai mô hình bốn biến đều có thể được sử dụng. Các mô hình có thể sẽ đáp ứng đủ nhu cầu.

Phương pháp chọn lọc từng bước tiến có thể được áp dụng ngay cả trong không gian đa chiều. thiết lập trong đó $n < p$; tuy nhiên, trong trường hợp này, chỉ có thể xây dựng các mô hình con $M_0, \dots, M_{n-1}$, vì mỗi mô hình con được hiệu chỉnh bằng phương pháp bình phương tối thiểu. Điều này sẽ không tạo ra một nghiệm duy nhất nếu $p \geq n$.

Lựa chọn từng bước ngược

Tương tự như phương pháp chọn lọc từng bước tiến, phương pháp chọn lọc từng bước lùi cũng cung cấp một

lùi lại
từng bước
lựa chọn

2Mặc dù phương pháp lựa chọn từng bước tiến xem xét $p(p + 1)/2 + 1$ mô hình, nhưng nó thực hiện tìm kiếm có hướng dẫn trên không gian mô hình, và do đó không gian mô hình hiệu quả được xem xét chứa nhiều hơn đáng kể so với các mô hình $p(p + 1)/2 + 1$.

Đây là một phương pháp thay thế hiệu quả cho việc lựa chọn tập con tốt nhất. Tuy nhiên, không giống như phương pháp lựa chọn từng bước tiến lên, nó bắt đầu với mô hình bình phương tối thiểu đầy đủ chứa tất cả p biến dự đoán, và sau đó loại bỏ dần biến dự đoán ít hữu ích nhất, từng biến một. Chi tiết được trình bày trong Thuật toán 6.3.

Thuật toán 6.3 Lựa chọn từng bước ngược
1. Gọi M_p là mô hình đầy đủ, bao gồm tất cả p biến dự đoán. 2. Với $k = p, p - 1, \dots, 1$: (a) Hãy xem xét tất cả k mô hình chứa tất cả các biến dự đoán trong M_k ngoại trừ một biến, với tổng số $k - 1$ biến dự đoán. (b) Chọn mô hình tốt nhất trong số k mô hình này và gọi nó là M_{k-1}. Ở đây, "tốt nhất" được định nghĩa là có RSS nhỏ nhất hoặc R^2 cao nhất. 3. Chọn một mô hình tốt nhất trong số $M_0, \dots, M_p$ bằng cách sử dụng sai số dự đoán trên tập dữ liệu kiểm định, C_p (AIC), BIC hoặc R^2 hiệu chỉnh. Hoặc sử dụng phương pháp kiểm định chéo.

Giống như phương pháp chọn lọc từng bước tiến, phương pháp chọn lọc từng bước lùi chỉ tìm kiếm qua $1 + p(p + 1)/2$ mô hình, và do đó có thể được áp dụng trong các trường hợp p quá lớn để áp dụng phương pháp chọn tập con tốt nhất.³ Cũng giống như phương pháp chọn lọc từng bước tiến, phương pháp chọn lọc từng bước lùi không đảm bảo sẽ cho ra mô hình tốt nhất chứa một tập con của p biến dự đoán.

Phương pháp chọn lọc ngược yêu cầu số lượng mẫu n lớn hơn số lượng biến p (để có thể xây dựng mô hình đầy đủ). Ngược lại, phương pháp chọn lọc từng bước tiến có thể được sử dụng ngay cả khi $n < p$, và do đó là phương pháp chọn tập con khả thi duy nhất khi p rất lớn.

Các phương pháp kết hợp

Các phương pháp chọn tập con tốt nhất, chọn từng bước tiến và chọn từng bước lùi thường cho ra các mô hình tương tự nhưng không hoàn toàn giống nhau. Một phương án khác là sử dụng các phiên bản lai giữa chọn từng bước tiến và chọn từng bước lùi, trong đó các biến được thêm vào mô hình một cách tuần tự, tương tự như chọn tiến. Tuy nhiên, sau khi thêm mỗi biến mới, phương pháp này cũng có thể loại bỏ bất kỳ biến nào không còn cải thiện độ phù hợp của mô hình. Cách tiếp cận này cố gắng mô phỏng sát hơn phương pháp chọn tập con tốt nhất trong khi vẫn giữ được lợi thế về tính toán của chọn từng bước tiến và chọn từng bước lùi.

6.1.3 Lựa chọn mô hình tối ưu

Việc lựa chọn tập con tốt nhất, lựa chọn tiến và lựa chọn lùi dẫn đến việc tạo ra một tập hợp các mô hình, mỗi mô hình chứa một tập con của p

³Giống như phương pháp chọn lọc từng bước tiến, phương pháp chọn lọc từng bước lùi thực hiện tìm kiếm có hướng dẫn trên không gian mô hình, và do đó xem xét hiệu quả nhiều hơn đáng kể so với $1 + p(p + 1)/2$ mô hình.

Các biến dự đoán. Để áp dụng các phương pháp này, chúng ta cần một cách để xác định mô hình nào là tốt nhất. Như đã thảo luận trong Phần 6.1.1, mô hình chứa tất cả các biến dự đoán sẽ luôn có RSS nhỏ nhất và R2 lớn nhất, vì các đại lượng này liên quan đến lỗi huấn luyện. Thay vào đó, chúng ta muốn chọn một mô hình có lỗi kiểm thử thấp. Như đã thấy rõ ở đây, và như chúng ta sẽ chỉ ra trong Chương 2, lỗi huấn luyện có thể là một ước lượng không chính xác về lỗi kiểm thử.

Do đó, RSS và R2 không phù hợp để chọn ra mô hình tốt nhất trong một tập hợp các mô hình có số lượng biến dự đoán khác nhau.

Để lựa chọn mô hình tốt nhất dựa trên sai số kiểm thử, chúng ta cần phải...

Ước tính sai số kiểm tra này. Có hai cách tiếp cận phổ biến:

- 1. Chúng ta có thể ước tính gián tiếp sai số kiểm thử bằng cách điều chỉnh sai số huấn luyện để tính đến sai lệch do hiện tượng quá khớp gây ra.
- 2. Chúng ta có thể ước tính trực tiếp sai số kiểm thử bằng cách sử dụng phương pháp tập dữ liệu xác thực hoặc phương pháp kiểm định chéo, như đã thảo luận trong Chương 5.

Chúng ta sẽ xem xét cả hai phương pháp này ở phần dưới đây.

Cp, AIC, BIC và R2 hiệu chỉnh

Chúng tôi đã chỉ ra trong Chương 2 rằng MSE của tập huấn luyện thường là ước lượng thấp hơn MSE của tập kiểm tra. (Nhớ lại rằng $MSE = RSS/n$.) Điều này là do khi chúng ta điều chỉnh mô hình cho dữ liệu huấn luyện bằng phương pháp bình phương nhỏ nhất, chúng ta ước lượng cụ thể các hệ số hồi quy sao cho RSS của tập huấn luyện (nhưng không phải RSS của tập kiểm tra) là nhỏ nhất có thể. Cụ thể, sai số huấn luyện sẽ giảm khi có nhiều biến hơn được đưa vào mô hình, nhưng sai số kiểm tra có thể không giảm. Do đó, RSS của tập huấn luyện và R2 của tập huấn luyện không thể được sử dụng để lựa chọn giữa một tập hợp các mô hình có số lượng biến khác nhau.

Tuy nhiên, có một số kỹ thuật để điều chỉnh lỗi huấn luyện cho kích thước mô hình. Các phương pháp này có thể được sử dụng để lựa chọn trong số một tập hợp các mô hình với số lượng biến khác nhau. Bây giờ chúng ta xem xét bốn phương pháp như vậy: Cp, tiêu chí thông tin Akaike (AIC), tiêu chí thông tin Bayes (BIC) và R2 hiệu chỉnh. Hình 6.2 hiển thị Cp, BIC và R2 hiệu chỉnh cho mô hình tốt nhất của mỗi kích thước được tạo ra bằng cách chọn tập con tốt nhất trên tập dữ liệu Credit.

Đối với mô hình bình phương tối thiểu phù hợp chứa d biến dự đoán, ước tính Cp

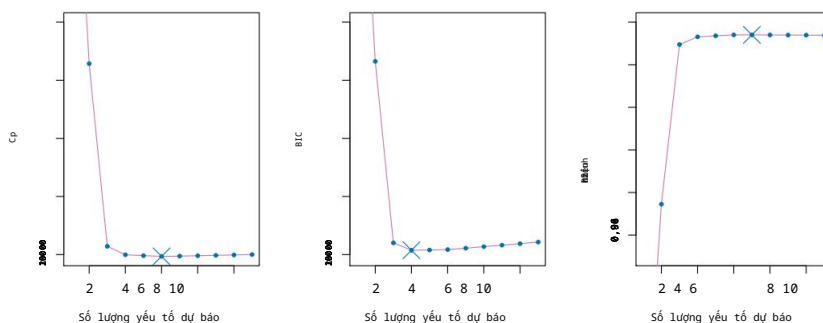
Sai số bình phương trung bình (MSE) của bài kiểm tra được tính bằng phương trình sau:

$$C_p = \frac{1}{N} RSS + 2\sigma^2 \quad , \quad (6.2)$$

trong đó σ^2 là ước tính phương sai của sai số liên quan đến mỗi phép đo phản hồi trong (6.1). Thông thường, σ^2 được ước tính bằng cách sử dụng mô hình đầy đủ chứa tất cả các biến dự đoán. Về cơ bản, thống kê Cp thêm một hình phạt $2\sigma^2$ vào RSS huấn luyện để điều chỉnh cho thực tế là sai số huấn luyện có xu hướng đánh giá thấp sai số kiểm tra. Rõ ràng, hình phạt tăng lên khi số lượng biến dự đoán trong mô hình tăng lên; điều này nhằm mục đích điều chỉnh

Cp của Mallows đôi khi được định nghĩa là $C_p = \frac{RSS}{\sigma^2} + 2d$. Điều này tương đương với Định nghĩa nêu trên theo nghĩa $C_p = \frac{1}{N} RSS + 2\sigma^2$, và do đó mô hình với Cp nhỏ nhất.

Cp
Tiêu
chí thông tin
Akaike
Thông tin
Bayes
tiêu chí
R2 đã điều chỉnh



HÌNH 6.2. C_p , BIC và R^2 hiệu chỉnh được hiển thị cho các mô hình tốt nhất của mỗi kích thước đối với tập dữ liệu **Tin dụng** (đường biên dưới trong Hình 6.1). C_p và BIC là các ước tính của MSE kiểm định. Trong biểu đồ ở giữa, chúng ta thấy rằng ước tính BIC về lỗi kiểm định cho thấy sự gia tăng sau khi chọn bốn biến. Hai biểu đồ còn lại khá bằng phẳng sau khi bao gồm bốn biến.

tương ứng với sự giảm RSS huấn luyện. Mặc dù nằm ngoài phạm vi của cuốn sách này, người ta có thể chứng minh rằng nếu σ^2 là ước lượng không thiên lệch của σ^2 trong (6.2), thì C_p là ước lượng không thiên lệch của MSE kiểm tra. Do đó, thống kê C_p có xu hướng có giá trị nhỏ đối với các mô hình có lỗi kiểm tra thấp, vì vậy khi xác định mô hình nào tốt nhất trong một tập hợp các mô hình, chúng ta chọn mô hình có giá trị C_p thấp nhất. Trong Hình 6.2, C_p chọn mô hình sáu biến chứa các biến dự đoán **income**, **limit**, **rating**, **cards**, **age** và **student**.

Tiêu chí AIC được định nghĩa cho một lớp lớn các mô hình được ước lượng bằng phương pháp hợp lý tối đa. Trong trường hợp mô hình (6.1) với sai số Gaussian, phương pháp hợp lý tối đa và phương pháp bình phương nhỏ nhất là giống nhau. Trong trường hợp này, AIC được đưa ra bởi

$$AIC = \frac{1}{N} \text{RSS} + 2d\sigma^2,$$

Trong đó, để đơn giản, chúng tôi đã bỏ qua các hằng số không liên quan.⁵ Do đó, đối với các mô hình bình phương tối thiểu, C_p và AIC tỷ lệ thuận với nhau, và vì vậy chỉ C_p được hiển thị trong Hình 6.2.

BIC được xây dựng dựa trên quan điểm Bayes, nhưng kết quả cuối cùng cũng tương tự như C_p (và AIC). Đối với mô hình bình phương tối thiểu với d biến dự đoán, BIC, ngoại trừ các hằng số không liên quan, được cho bởi

$$BIC = \frac{1}{N} \text{RSS} + \log(n)d\sigma^2. \quad (6.3)$$

Tương tự như C_p , BIC có xu hướng có giá trị nhỏ đối với mô hình có lỗi kiểm định thấp, vì vậy nói chung chúng ta chọn mô hình có giá trị BIC thấp nhất. Lưu ý rằng BIC thay thế $2d\sigma^2$ được sử dụng bởi C_p bằng một số hạng $\log(n)d\sigma^2$, trong đó n là số lượng quan sát. Vì $\log n > 2$ đối với bất kỳ $n > 7$ nào,

Có hai công thức tính AIC cho hồi quy bình phương tối thiểu. Công thức mà chúng tôi cung cấp ở đây yêu cầu một biểu thức cho σ^2 , mà chúng tôi thu được bằng cách sử dụng mô hình đầy đủ chứa tất cả các biến dự báo. Công thức thứ hai phù hợp khi σ^2 chưa biết và chúng ta không muốn ước tính nó một cách rõ ràng; công thức đó có một số hạng $\log(\text{RSS})$ thay vì số hạng RSS. Việc chứng minh chi tiết hai công thức này nằm ngoài phạm vi của cuốn sách này.

Chỉ số BIC thường đặt mức phạt nặng hơn đối với các mô hình có nhiều biến, do đó dẫn đến việc lựa chọn các mô hình nhỏ hơn so với C_p .

Trong Hình 6.2, chúng ta thấy rằng điều này thực sự đúng với tập dữ liệu *Tín dụng*; BIC chọn một mô hình chỉ chứa bốn biến dự báo: *thu nhập*, *hạn mức*, *thẻ* và *sinh viên*. Trong trường hợp này, các đường cong rất phẳng, do đó dường như không có nhiều sự khác biệt về độ chính xác giữa mô hình bốn biến và mô hình sáu biến.

Chỉ số R^2 hiệu chỉnh là một phương pháp phổ biến khác để lựa chọn giữa một tập hợp các mô hình có số lượng biến khác nhau. (Nhớ lại từ...)

Chương 3 nêu rằng R^2 thông thường được định nghĩa là $1 - \frac{RSS}{TSS}$, trong đó $TSS = (y_i - \bar{y})^2$ là tổng bình phương sai số của biến phản hồi. Vì RSS luôn giảm khi thêm nhiều biến vào mô hình, nên R^2 luôn tăng khi thêm nhiều biến. Đối với mô hình bình phương tối thiểu với d biến, thống kê R^2 hiệu chỉnh được tính như sau:

$$R^2 \text{ hiệu chỉnh} = 1 - \frac{RSS/(n - d - 1)}{TSS/(n - 1)} \quad (6.4)$$

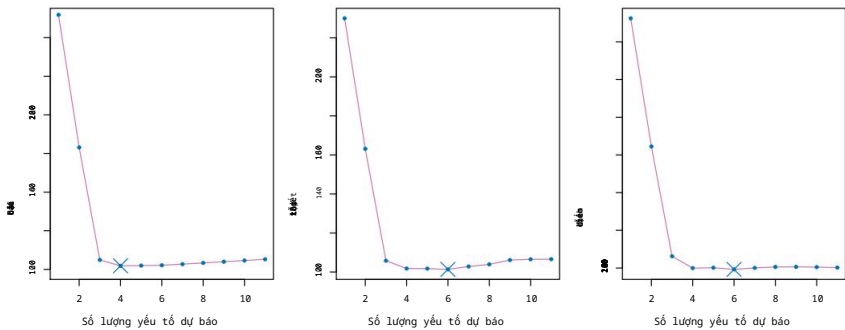
Không giống như C_p , AIC và BIC, trong đó giá trị nhỏ cho thấy mô hình có sai số kiểm định thấp, giá trị lớn của R^2 hiệu chỉnh cho thấy mô hình có sai số kiểm định nhỏ. Tối đa hóa R^2 hiệu chỉnh tương đương với việc tối thiểu hóa $n - d - 1$. Trong khi RSS luôn giảm khi số lượng biến trong mô hình $n - d - 1$ có thể tăng hoặc giảm, do sự hiện diện của $\frac{RSS}{n - d - 1}$ tăng lên, $\frac{RSS}{n - d - 1}$ trong mô hình mẫu số.

Ý tưởng đằng sau hệ số R^2 hiệu chỉnh là khi tất cả các biến chính xác đã được đưa vào mô hình, việc thêm các biến nhiễu bổ sung sẽ chỉ dẫn đến sự giảm rất nhỏ trong RSS. Vì việc thêm các biến nhiễu dẫn đến sự tăng lên của d , nên các biến này sẽ dẫn đến sự tăng lên của và do đó làm giảm R^2 hiệu chỉnh. Vì vậy, về lý thuyết, mô hình có $\frac{RSS}{n - d - 1}$, R^2 hiệu chỉnh lớn nhất sẽ chỉ có các biến chính xác và không có biến nhiễu. Không giống như thống kê R^2 , thống kê R^2 hiệu chỉnh phải trả giá cho việc đưa các biến không cần thiết vào mô hình. Hình 6.2 hiển thị R^2 hiệu chỉnh cho tập dữ liệu *Credit*. Sử dụng thống kê này dẫn đến việc lựa chọn một mô hình chứa bảy biến, bổ sung thêm vào mô hình được chọn bởi C_p và AIC.

C_p , AIC và BIC đều có những cơ sở lý thuyết chặt chẽ nằm ngoài phạm vi cuốn sách này. Những cơ sở lý thuyết này dựa trên các lập luận tiệm cận (các trường hợp kích thước mẫu n rất lớn). Mặc dù phổ biến và khá trực quan, R^2 hiệu chỉnh không được chứng minh rõ ràng trong lý thuyết thống kê như AIC, BIC và C_p . Tất cả các chỉ số này đều dễ sử dụng và tính toán. Ở đây, chúng tôi đã trình bày công thức của chúng trong trường hợp mô hình tuyến tính được hiệu chỉnh bằng phương pháp bình phương nhỏ nhất; tuy nhiên, AIC và BIC cũng có thể được định nghĩa cho các loại mô hình tổng quát hơn.

Xác thực và kiểm định chéo

Như một phương án thay thế cho các phương pháp vừa được thảo luận, chúng ta có thể trực tiếp ước tính lỗi kiểm thử bằng cách sử dụng tập dữ liệu xác thực và phương pháp kiểm định chéo đã được thảo luận trong Chương 5. Chúng ta có thể tính toán lỗi của tập dữ liệu xác thực hoặc lỗi kiểm định chéo cho mỗi mô hình đang được xem xét, và sau đó lựa chọn.



HÌNH 6.3. Đối với tập dữ liệu *Tin dụng*, ba đại lượng được hiển thị cho mô hình tốt nhất chứa d biến dự đoán, với d nằm trong khoảng từ 1 đến 11. Mô hình tốt nhất tổng thể dựa trên mỗi đại lượng này, được thể hiện bằng một dấu thập màu xanh lam. Bên trái: Hình vuông Góc của BIC. Trung tâm: Lỗi tập dữ liệu xác thực. Bên phải: Lỗi kiểm định chéo.

Mô hình mà kết quả sai số kiểm định ước tính nhỏ nhất. Quy trình này có ưu điểm hơn so với AIC, BIC, C_p và R^2 hiệu chỉnh ở chỗ: Phương pháp này cung cấp ước tính trực tiếp về sai số kiểm định và đưa ra ít giả định hơn. Về mô hình cơ bản thực sự. Nó cũng có thể được sử dụng trong nhiều lĩnh vực rộng hơn. các nhiệm vụ lựa chọn mô hình, ngay cả trong trường hợp khó xác định chính xác mô hình. bậc tự do (ví dụ: số lượng biến dự đoán trong mô hình) hoặc cứng để ước tính phương sai lỗi σ^2 . Lưu ý rằng khi sử dụng phương pháp kiểm định chéo, Trình tự các mô hình M_k trong Thuật toán 6.1-6.3 được xác định riêng biệt. Đối với mỗi fold huấn luyện, sai số xác thực được tính trung bình trên tất cả các fold. cho mỗi kích thước mô hình k . Điều này có nghĩa là, ví dụ với hồi quy tập con tốt nhất, M_k , tập con tốt nhất có kích thước k , có thể khác nhau giữa các lần gấp. Một khi Sau khi chọn kích thước k tối ưu, chúng ta tìm mô hình tốt nhất có kích thước đó trên toàn bộ dữ liệu. bộ.

Trước đây, việc thực hiện kiểm định chéo đòi hỏi chi phí tính toán rất lớn. đối với nhiều bài toán có p lớn và/hoặc n lớn, và do đó AIC, BIC, C_p , và hệ số R^2 hiệu chỉnh là những phương pháp hấp dẫn hơn để lựa chọn giữa các phương pháp khác nhau. tập hợp các mô hình. Tuy nhiên, ngày nay với máy tính tốc độ cao, các phép tính Các yêu cầu cần thiết để thực hiện kiểm định chéo hầu như không bao giờ là vấn đề. Do đó, kiểm định chéo là một phương pháp rất hấp dẫn để lựa chọn từ một số lượng lớn các ứng viên. các mô hình đang được xem xét.

Hình 6.3 hiển thị, theo hàm của d , BIC, lỗi tập dữ liệu xác thực và Sai số kiểm định chéo trên dữ liệu *Tin dụng*, đối với mô hình d -biến tốt nhất. Các lỗi xác thực được tính toán bằng cách chọn ngẫu nhiên ba phần tư. Một phần các quan sát được dùng làm tập huấn luyện, và phần còn lại làm tập kiểm chứng. Sai số kiểm chứng chéo được tính toán bằng cách sử dụng $k = 10$ fold. Trong trường hợp này, cả phương pháp xác thực và phương pháp kiểm định chéo đều cho ra kết quả là mô hình sáu biến. Tuy nhiên, cả ba phương pháp đều cho thấy rằng mô hình bốn biến, Các mô hình năm và sáu biến về cơ bản là tương đương nhau về mặt kiểm định. lỗi.

Trên thực tế, các đường cong sai số kiểm định ước tính hiển thị ở giữa và bên phải của Hình 6.3 khá phẳng. Trong khi mô hình ba biến rõ ràng có sai số kiểm định ước tính thấp hơn so với mô hình hai biến, kiểm định ước tính Sai số của các mô hình từ 3 đến 11 biến khá giống nhau. Hơn nữa, nếu chúng ta

Lặp lại phương pháp tập dữ liệu xác thực bằng cách chia dữ liệu thành các phần khác nhau. một tập huấn luyện và một tập kiểm định, hoặc nếu chúng ta lặp lại quá trình kiểm định chéo bằng cách sử dụng một tập hợp các fold kiểm định chéo khác nhau, sau đó là mô hình chính xác với Sai số kiểm tra ước tính thấp nhất chắc chắn sẽ thay đổi. Trong trường hợp này, chúng ta có thể Chọn mô hình bằng cách sử dụng quy tắc một sai số chuẩn. Trước tiên, chúng ta tính toán sai số chuẩn của MSE kiểm định ước tính cho mỗi kích thước mô hình, và sau đó Chọn mô hình nhỏ nhất mà sai số kiểm định ước tính nằm trong phạm vi một.

sai số chuẩn của điểm thấp nhất trên đường cong. Lý do ở đây là... Nếu một tập hợp các mô hình có vẻ tốt ngang nhau, thì chúng ta có thể Tốt nhất nên chọn mô hình đơn giản nhất—tức là mô hình có kích thước nhỏ nhất. số lượng biến dự đoán. Trong trường hợp này, áp dụng quy tắc một sai số chuẩn. Việc sử dụng tập dữ liệu kiểm định hoặc phương pháp kiểm định chéo dẫn đến việc lựa chọn... mô hình ba biến.

một-
tiêu chuẩn-
lỗi
luật lệ

6.2 Phương pháp co ngót

Các phương pháp lựa chọn tập con được mô tả trong Phần 6.1 liên quan đến việc sử dụng ít nhất các hình vuông được sử dụng để phù hợp với mô hình tuyến tính chứa một tập hợp con các biến dự đoán. Như một Ngoài ra, chúng ta có thể xây dựng một mô hình chứa tất cả p biến dự đoán bằng cách sử dụng một kỹ thuật. điều đó hạn chế hoặc chuẩn hóa các ước tính hệ số, hay nói cách khác, điều đó Làm giảm ước lượng hệ số về phía 0. Có thể không ngay lập tức Rõ ràng là ràng buộc như vậy sẽ cải thiện độ phù hợp, nhưng hóa ra là Việc thu hẹp các ước tính hệ số có thể làm giảm đáng kể phương sai của chúng. Hai kỹ thuật nổi tiếng nhất để thu nhỏ các hệ số hồi quy là... Các phương pháp hướng về số 0 bao gồm hồi quy Ridge và Lasso.

6.2.1 Hồi quy Ridge

Hãy nhớ lại từ Chương 3 rằng quy trình khớp bình phương nhỏ nhất ước tính $\beta_0, \beta_1, \dots, \beta_p$ sử dụng các giá trị tối thiểu hóa

$$RSS = \sum_{i=1}^N y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \quad .$$

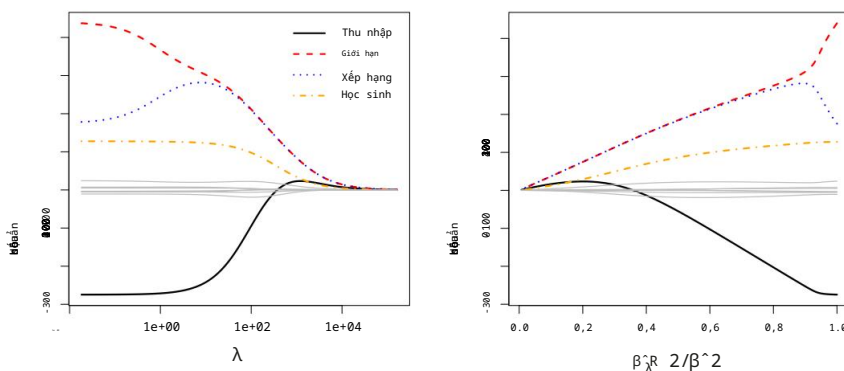
Hồi quy Ridge rất giống với phương pháp bình phương tối thiểu, ngoại trừ việc các hệ số được ước tính bằng cách tối thiểu hóa một đại lượng hơi khác. Cụ thể, Các ước lượng hệ số hồi quy Ridge β^*R là các giá trị tối thiểu hóa

sườn núi
hồi quy

$$\sum_{i=1}^N y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \quad + \lambda \sum_{j=1}^p \beta_j^2 = RSS + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.5)$$

trong đó $\lambda \geq 0$ là tham số điều chỉnh, được xác định riêng. Phương trình 6.5 cân bằng giữa hai tiêu chí khác nhau. Cũng như phương pháp bình phương tối thiểu, hồi quy Ridge tìm kiếm các ước lượng hệ số phù hợp với dữ liệu tốt, bằng cách tạo ra RSS nhỏ. Tuy nhiên, số hạng thứ hai, $\lambda \sum_{j=1}^p \beta_j^2$, được gọi là hình phạt giảm giá trị, là sự nhỏ khi $\beta_1, \dots, \beta_p$ gần bằng 0, và do đó nó có tác dụng làm giảm kích thước. ước tính của β_j về 0. Tham số điều chỉnh λ dùng để kiểm soát

điều chỉnh
tham số
giảm giá trị
phạt các



HÌNH 6.4. Các hệ số hồi quy Ridge được chuẩn hóa được hiển thị cho Tập dữ liệu *Tín dụng*, như một hàm của λ và $\beta^2 R / \beta^2$.

Tác động tương đối của hai thuật ngữ này lên các ước tính hệ số hồi quy. Khi $\lambda = 0$, thuật ngữ phạt không có tác dụng, và hồi quy Ridge sẽ tạo ra các ước tính bình phương nhỏ nhất. Tuy nhiên, khi $\lambda \rightarrow \infty$, tác động của mức phạt co rút tăng lên, và các ước tính hệ số hồi quy Ridge sẽ tiến đến 0. Không giống như phương pháp bình phương tối thiểu, chỉ tạo ra một tập hợp các ước lượng hệ số, hồi quy Ridge sẽ tạo ra một tập hợp các hệ số khác nhau. Ước tính, $\beta^2 R / \beta^2$, với mỗi giá trị của λ . Việc lựa chọn một giá trị tốt cho λ là rất quan trọng; Chúng tôi sẽ hoàn thảo luận vấn đề này đến Mục 6.2.3, nơi chúng tôi sử dụng phương pháp kiểm định chéo.

Lưu ý rằng trong (6.5), hình phạt co rút được áp dụng cho $\beta_1, \dots, \beta_p$, nhưng không phải là hệ số chặn β_0 . Chúng ta muốn thu hẹp mối liên hệ ước tính của mỗi biến với phản hồi; tuy nhiên, chúng ta không muốn thu nhỏ hệ số chặn, đơn giản chỉ là thước đo giá trị trung bình của phản hồi.

khi $x_{i1} = x_{i2} = \dots = x_{ip} = 0$. Nếu ta giả sử rằng các biến số—tức là, Các cột của ma trận dữ liệu X đã được căn giữa để có giá trị trung bình bằng không.

Trước khi thực hiện hồi quy Ridge, hệ số chặn ước tính sẽ được lấy dạng $\beta^2 R / \beta^2 = \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$.

Đơn xin cấp dữ liệu tín dụng

Hình 6.4 thể hiện ước tính hệ số hồi quy Ridge cho dữ liệu *Tín dụng*. Các tập hợp được hiển thị. Ở bảng bên trái, mỗi đường cong tương ứng với... Ước tính hệ số hồi quy Ridge cho một trong mười biến số, được vẽ đồ thị như một hàm của λ . Ví dụ, đường liền nét màu đen biểu thị đường gờ. Ước tính hồi quy cho hệ số *thu nhập*, khi λ thay đổi. Ở điểm cực trị ở phía bên trái của đồ thị, λ về cơ bản bằng 0, và do đó tương ứng Các ước lượng hệ số Ridge giống với các ước lượng bình phương tối thiểu thông thường. Nhưng khi λ tăng, các ước lượng hệ số Ridge thu hẹp lại. bằng không. Khi λ cực kỳ lớn, thì tất cả các ước tính hệ số sườn dốc đều bằng không. Về cơ bản là bằng không; điều này tương ứng với mô hình trống không chứa biến dự báo nào. Trong biểu đồ này, các biến *thu nhập*, *giới hạn*, *xếp hạng* và *sinh viên* là được hiển thị bằng các màu sắc khác nhau, vì các biến này có xu hướng có ảnh hưởng lớn nhất. ước tính hệ số lớn nhất. Trong khi đó, các ước tính hệ số Ridge có xu hướng tổng thể giảm khi λ tăng, các hệ số riêng lẻ, chẳng hạn như *xếp hạng*, cũng giảm. và *thu nhập*, đôi khi có thể tăng lên khi λ tăng.

Bảng bên phải của Hình 6.4 hiển thị cùng một hệ số giờ. Các ước tính được thể hiện ở bảng bên trái, nhưng thay vì hiển thị λ trên trục x, Bây giờ chúng ta hiển thị β^2 , trong đó β^2 biểu thị vectơ bình phương tối thiểu. ước lượng hệ số. Ký hiệu β^2 biểu thị chuẩn 2 (đọc là beta 2). Nó đo lường

"ell 2") của một vectơ, và được định nghĩa là $\beta^2 = \sum_{j=1}^p$ khoảng cách của β so với số 0. Khi λ tăng, nó sẽ luôn giảm, và do đó β^2 cũng sẽ giảm. β^2 nằm trong khoảng từ 1 (khi $\lambda = 0$, trong trường hợp đó ước tính hệ số hồi quy Ridge là như nhau. như ước lượng bình phương nhỏ nhất, và do đó (khi $\lambda = 0$ chuẩn mực giống nhau) đến ∞ , trong trường hợp đó ước lượng hệ số hồi quy Ridge là một vectơ gồm các số 0, với chuẩn 2 bằng 0). Do đó, chúng ta có thể nghĩ đến trục x ở bảng bên phải của Hình 6.4 biểu thị lượng mà đường gờ Các ước tính hệ số hồi quy đã bị thu hẹp về phía 0; một lượng nhỏ Giá trị này cho thấy chúng đã bị thu nhỏ rất gần bằng không.

Các ước lượng hệ số bình phương tối thiểu tiêu chuẩn được thảo luận trong Chương 3. có tính chất bất biến theo tỷ lệ: nhân X_j với một hằng số c đơn giản chỉ dẫn đến việc điều chỉnh tỷ lệ các ước tính hệ số bình phương nhỏ nhất theo hệ số 1/c. Trong các trường hợp khác, Nói cách khác, bất kể biến dự đoán thứ j được điều chỉnh tỷ lệ như thế nào, $X_j \beta_j$ vẫn sẽ giữ nguyên. Tương tự. Ngược lại, các ước tính hệ số hồi quy Ridge có thể thay đổi đáng kể khi nhân một biến dự báo nhất định với một hằng số. Ví dụ, Hãy xem xét biến số thu nhập, được đo bằng đô la. Người ta hoàn toàn có thể đo thu nhập bằng nghìn đô la, điều này sẽ dẫn đến... trong đó giá trị thu nhập quan sát được giảm đi 1.000 lần. Hiện nay đến hạn vào tổng bình phương hệ số trong công thức hồi quy Ridge (6.5), sự thay đổi về quy mô như vậy sẽ không chỉ đơn giản làm cho ước tính hệ số hồi quy Ridge cho thu nhập thay đổi theo hệ số 1.000. Nói cách khác, $X_j \beta_j$ sẽ phụ thuộc không chỉ vào giá trị của λ , mà còn vào tỷ lệ của Biến dự đoán thứ j. Trên thực tế, giá trị của $X_j \beta_j$ thậm chí có thể phụ thuộc vào tỷ lệ của các biến dự báo khác ! Do đó, tốt nhất nên áp dụng hồi quy Ridge sau chuẩn hóa các biến dự báo, sử dụng công thức

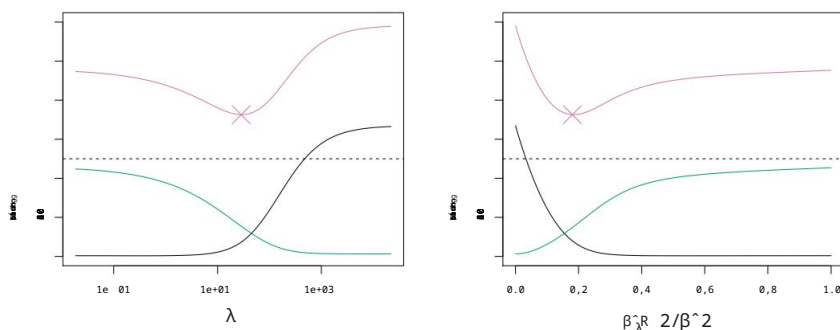
$$x_{ij}^* = \frac{x_{ij}}{\sqrt{\frac{1}{N} \sum_{i=1}^N (x_{ij} - \bar{x}_j)^2}}, \tag{6.6}$$

để tất cả chúng đều ở cùng một thang đo. Trong (6.6), mẫu số là độ lệch chuẩn ước tính của biến dự báo thứ j. Do đó, tất cả các biến dự báo được chuẩn hóa sẽ có độ lệch chuẩn bằng một. Do đó, kết quả phù hợp cuối cùng sẽ không phụ thuộc vào thang đo mà các biến dự báo được sử dụng. được đo. Trong Hình 6.4, trục y hiển thị các ước tính hệ số hồi quy Ridge được chuẩn hóa – tức là các ước tính hệ số thu được từ Thực hiện hồi quy Ridge bằng cách sử dụng các biến dự báo được chuẩn hóa.

Tại sao phương pháp hồi quy Ridge lại cho kết quả tốt hơn phương pháp bình phương tối thiểu?

Ưu điểm của hồi quy Ridge so với phương pháp bình phương tối thiểu nằm ở độ lệch-phương sai. Sự đánh đổi. Khi λ tăng, tính linh hoạt của phép hồi quy Ridge giảm đi. Điều này dẫn đến giảm phương sai nhưng tăng độ lệch. Điều này được minh họa trong Bảng bên trái của Hình 6.5, sử dụng bộ dữ liệu mô phỏng chứa $p = 45$ các biến dự báo và $n = 50$ quan sát. Đường cong màu xanh lá cây ở bảng bên trái.

6.2 Phương pháp co ngót 243



HÌNH 6.5. Sai số bình phương (màu đen), phương sai (màu xanh lá cây) và bình phương trung bình của phép thử lỗi (màu tím) đối với các dự đoán hồi quy Ridge trên tập dữ liệu mô phỏng, như một hàm của λ và $\beta^R \lambda^2 / \beta^2$. Các đường chấm ngang biểu thị điểm cực tiểu. MSE có thể xảy ra. Các dấu thập màu tím biểu thị các mô hình hồi quy Ridge mà MSE là nhỏ nhất.

Hình 6.5 hiển thị phương sai của các dự đoán hồi quy Ridge như một hàm của λ . Tại các ước tính hệ số bình phương nhỏ nhất, tương ứng với Đối với hồi quy Ridge với $\lambda = 0$, phương sai cao nhưng không có độ lệch. Nhưng Khi λ tăng, sự co lại của các ước tính hệ số Ridge dẫn đến giảm đáng kể độ biến thiên của các dự đoán, nhưng phải trả giá bằng một Sự gia tăng nhẹ về độ lệch. Hãy nhớ rằng sai số bình phương trung bình (MSE) của phép thử, được biểu diễn bằng màu tím, có liên quan chặt chẽ đến phương sai cộng với bình phương độ lệch. Đối với Với các giá trị của λ lên đến khoảng 10, phương sai giảm nhanh chóng, với rất ít Sự gia tăng độ lệch được thể hiện bằng đường màu đen. Do đó, MSE giảm đáng kể khi λ tăng từ 0 đến 10. Vượt quá điểm này, phương sai giảm.

do λ tăng nên quá trình chậm lại, và sự co rút trên các hệ số gây ra chúng bị đánh giá thấp đáng kể, dẫn đến sự gia tăng lớn về độ sai lệch.

Sai số bình phương trung bình (MSE) tối thiểu đạt được ở mức xấp xỉ $\lambda = 30$. Điều thú vị là, do phương sai cao, MSE liên quan đến phương pháp bình phương tối thiểu Độ phù hợp, khi $\lambda = 0$, gần như cao bằng độ phù hợp của mô hình null mà trong đó tất cả Các ước tính hệ số bằng 0 khi $\lambda = \infty$. Tuy nhiên, đối với một giá trị trung gian Với giá trị của λ , MSE sẽ thấp hơn đáng kể.

Hình 6.5, bảng bên phải, hiển thị các đường cong tương tự như bảng bên trái, lần này được vẽ dựa trên chuẩn 2 của hồi quy Ridge.

ước lượng hệ số chia cho chuẩn 2 của ước lượng bình phương nhỏ nhất.

Khi chúng ta di chuyển từ trái sang phải, các khớp nối trở nên linh hoạt hơn, và do đó Độ lệch giảm và phương sai tăng.

Nhìn chung, trong những tình huống mà mối quan hệ giữa phản hồi và các biến dự báo gần như tuyến tính, các ước tính bình phương nhỏ nhất sẽ có Độ lệch thấp nhưng có thể có phương sai cao. Điều này có nghĩa là một thay đổi nhỏ trong Dữ liệu huấn luyện có thể gây ra sự thay đổi lớn trong hệ số bình phương tối thiểu. ước tính. Đặc biệt, khi số lượng biến p gần bằng lớn.

như số lượng quan sát n , như trong ví dụ ở Hình 6.5,

Các ước tính bình phương nhỏ nhất sẽ có độ biến thiên rất lớn. Và nếu $p > n$ thì

Các ước lượng bình phương tối thiểu thậm chí không có nghiệm duy nhất, trong khi phương pháp Ridge lại khác.

Phương pháp hồi quy vẫn có thể hoạt động tốt bằng cách đánh đổi một chút độ lệch để có được một mức độ chính xác nhất định.

sự giảm đáng kể về phương sai. Do đó, hồi quy Ridge hoạt động tốt nhất trong các trường hợp này. trong đó các ước lượng bình phương nhỏ nhất có phương sai cao.

Hồi quy Ridge cũng có những ưu điểm vượt trội về mặt tính toán so với các phương pháp tốt nhất. Việc lựa chọn tập con, đôi khi phải tìm kiếm thông qua các mô hình 2^p . Như chúng ta đã thảo luận trước đó, ngay cả với các giá trị p vừa phải, việc tìm kiếm như vậy cũng có thể về mặt tính toán là không khả thi. Ngược lại, với bất kỳ giá trị λ cố định nào, hồi quy Ridge chỉ phù hợp với một mô hình duy nhất, và quy trình phù hợp mô hình có thể được thực hiện khá nhanh. Trên thực tế, người ta có thể chứng minh rằng các phép tính cần thiết để giải (6.5), đồng thời cho tất cả các giá trị của λ , gần như giống hệt nhau. đến những người sử dụng phương pháp bình phương tối thiểu để hiệu chỉnh mô hình.

6.2.2 Dây thừng

Hồi quy Ridge có một nhược điểm rõ ràng. Không giống như tập con tốt nhất, lựa chọn từng bước tiến và lựa chọn từng bước lùi, điều này thường sẽ Chọn các mô hình chỉ bao gồm một tập hợp con các biến, hồi quy Ridge sẽ bao gồm tất cả p biến dự đoán trong mô hình cuối cùng. Hình phạt $\lambda \beta_j^2$ trong (6.5) Nó sẽ thu nhỏ tất cả các hệ số về 0, nhưng sẽ không thiết lập bất kỳ hệ số nào trong số đó. chính xác bằng không (trừ khi $\lambda = \infty$). Điều này có thể không phải là vấn đề đối với dự đoán. độ chính xác cao, nhưng điều này có thể tạo ra thách thức trong việc giải thích mô hình trong các thiết lập khác nhau. trong đó số lượng biến p khá lớn. Ví dụ, trong Credit Dựa trên tập dữ liệu này, có vẻ như các biến quan trọng nhất là thu nhập, giới hạn, xếp hạng và học sinh. Vì vậy, chúng ta có thể muốn xây dựng một mô hình chỉ bao gồm những yếu tố dự báo này. Tuy nhiên, hồi quy Ridge sẽ luôn tạo ra một mô hình. liên quan đến cả mười biến dự báo. Việc tăng giá trị của λ sẽ có xu hướng làm giảm độ lớn của các hệ số, nhưng sẽ không dẫn đến việc loại trừ bất kỳ hệ số nào. các biến số.

Lasso là một phương pháp thay thế tương đối mới cho hồi quy Ridge, khắc phục được nhược điểm này. Các hệ số Lasso, β^L , giảm thiểu số lượng

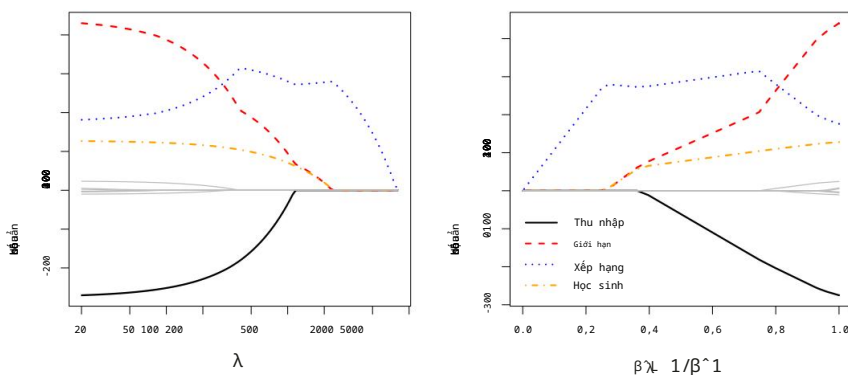
dây thừng

$$\sum_{i=1}^N y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|. \tag{6.7}$$

So sánh (6.7) với (6.5), chúng ta thấy rằng hồi quy lasso và ridge có Công thức tương tự. Sự khác biệt duy nhất là β_j^2 thuật ngữ trong dây nịt Hình phạt hồi quy (6.5) đã được thay thế bằng $|\beta_j|$ trong hình phạt lasso (6.7). Theo thuật ngữ thống kê, thuật toán lasso sử dụng hình phạt 1 (phát âm là “ell 1”). thay vì mức phạt 2. Chuẩn 1 của vectơ hệ số β được cho bởi $\beta_1 = \sum |\beta_j|$.

Tương tự như hồi quy Ridge, phương pháp Lasso thu hẹp các ước tính hệ số về phía 0. Tuy nhiên, trong trường hợp của Lasso, hình phạt 1 có tác dụng... bước một số ước tính hệ số phải bằng chính xác không khi Tham số điều chỉnh λ đủ lớn. Do đó, tương tự như phương pháp chọn tập con tốt nhất, thuật toán Lasso thực hiện việc chọn biến. Kết quả là, các mô hình được tạo ra Các kết quả từ phương pháp lasso thường dễ hiểu hơn nhiều so với các kết quả được tạo ra bằng phương pháp thủ công. bằng phương pháp hồi quy Ridge. Chúng ta nói rằng phương pháp Lasso tạo ra các mô hình thưa thớt – nghĩa là, các mô hình chỉ bao gồm một tập hợp con các biến. Cũng như trong hồi quy Ridge, Việc lựa chọn giá trị λ phù hợp cho thuật toán lasso là rất quan trọng; chúng ta sẽ tạm hoãn thảo luận về vấn đề này. Xem Mục 6.2.3, nơi chúng tôi sử dụng phương pháp kiểm định chéo.

thưa thớt



HÌNH 6.6. Các hệ số lasso chuẩn hóa trên tập dữ liệu *Tín dụng* là được biểu diễn dưới dạng hàm của λ và $\beta\lambda / 1/\beta^1$.

Ví dụ, hãy xem xét các biểu đồ hệ số trong Hình 6.6, được tạo ra bằng cách áp dụng thuật toán lasso cho tập dữ liệu *Tín dụng*. Khi $\lambda = 0$, thì Phương pháp lasso chỉ đơn giản là đưa ra kết quả khớp bình phương nhỏ nhất, và khi λ trở nên đủ lớn. Với kích thước lớn, lasso đưa ra mô hình null trong đó tất cả các ước tính hệ số đều bằng nhau. số không. Tuy nhiên, nằm giữa hai thái cực này, hồi quy Ridge và Các mô hình lasso khá khác nhau. Di chuyển từ trái sang phải. Trong bảng bên phải của Hình 6.6, chúng ta quan sát thấy ban đầu kết quả của thuật toán lasso chỉ tạo ra một mô hình chứa biến dự đoán *xếp hạng*. Sau đó, *sinh viên* và *Giới hạn* được đưa vào mô hình gần như đồng thời, ngay sau đó là *thu nhập*. Cuối cùng, các biến còn lại sẽ được đưa vào mô hình. Do đó, tùy thuộc vào... Với giá trị của λ , phương pháp Lasso có thể tạo ra mô hình bao gồm bất kỳ số lượng biến nào. Ngược lại, hồi quy Ridge sẽ luôn bao gồm tất cả các biến trong mô hình, mặc dù độ lớn của các ước tính hệ số sẽ phụ thuộc vào trên λ .

Một công thức khác cho hồi quy Ridge và Lasso

Người ta có thể chứng minh rằng các ước lượng hệ số hồi quy lasso và ridge giải quyết được vấn đề các vấn đề

$$\begin{aligned} & \text{giảm thiểu} \quad \sum_{i=1}^N y_i^2 - 2 \sum_{j=1}^p \beta_j x_{ij} y_i + \sum_{j=1}^p \beta_j^2 \\ & \text{tùy thuộc vào} \quad |\beta_j| \leq s \end{aligned} \quad (6.8)$$

Và

$$\begin{aligned} & \text{giảm thiểu} \quad \sum_{i=1}^N y_i^2 - 2 \sum_{j=1}^p \beta_j x_{ij} y_i + \sum_{j=1}^p \beta_j^2 \\ & \text{tùy thuộc vào} \quad \beta_j^2 \leq s, \end{aligned} \quad (6.9)$$

tương ứng. Nói cách khác, với mỗi giá trị của λ , tồn tại một số s sao cho Các phương trình (6.7) và (6.8) sẽ cho cùng một ước tính hệ số lasso. Tương tự, với mỗi giá trị của λ đều có một s tương ứng sao cho Phương trình (6.5) và (6.9) sẽ cho cùng một ước tính hệ số hồi quy Ridge.

Khi $p = 2$, thì (6.8) cho biết các ước tính hệ số lasso có RSS nhỏ nhất trong tất cả các điểm nằm trong hình thoi được xác định bởi $|\beta_1| + |\beta_2| \leq s$. Tương tự, các ước tính hồi quy Ridge có giá trị nhỏ nhất. RSS từ tất cả các điểm nằm trong vòng tròn được xác định bởi $\beta_1^2 + \beta_2^2 \leq s$. Chúng ta có thể nghĩ về (6.8) như sau. Khi chúng ta thực hiện động tác lasso, chúng ta đang cố gắng tìm tập hợp các ước lượng hệ số dẫn đến RSS nhỏ nhất, tuân theo với ràng buộc rằng có một ngân sách s quy định độ lớn của $|\beta_j|$. Khi s cực kỳ lớn, thì ngân sách này không quá hạn chế, và do đó các ước lượng hệ số có thể lớn. Trên thực tế, nếu s đủ lớn thì Nếu giải pháp bình phương nhỏ nhất nằm trong phạm vi ngân sách, thì (6.8) sẽ đơn giản là giải pháp bình phương tối thiểu. Ngược lại, nếu s nhỏ thì $|\beta_j|$ phải là nhỏ để tránh vi phạm ngân sách. Tương tự, (6.9) chỉ ra rằng Khi thực hiện hồi quy Ridge, chúng ta tìm kiếm một tập hợp các ước lượng hệ số sao cho RSS càng nhỏ càng tốt, tuân theo yêu cầu rằng $\sum_{j=1}^p \beta_j^2$ không vượt quá ngân sách. Các công thức (6.8) và (6.9) cho thấy mối liên hệ chặt chẽ giữa Lasso, hồi quy Ridge và lựa chọn tập con tốt nhất. Hãy xem xét vấn đề này.

giảm thiểu

β

$\sum_{i=1}^n y_i$

β_0

$\sum_{j=1}^p \beta_j x_{ij}$

β_0

tùy thuộc vào

$\sum_{j=1}^p \beta_j^2$

$I(\beta_j = 0) \leq s$

(6.10)

Ở đây $I(\beta_j = 0)$ là một biến chỉ thị: nó nhận giá trị bằng 1 nếu $\beta_j = 0$, và bằng không trong trường hợp khác. Khi đó (6.10) tương đương với việc tìm một tập hợp các hệ số ước tính sao cho RSS nhỏ nhất có thể, tuân theo ràng buộc rằng không có quá s hệ số có thể khác 0. Vấn đề (6.10) là tương đương với việc lựa chọn tập con tốt nhất. Thật không may, việc giải (6.10) là không khả thi về mặt tính toán khi p lớn, vì nó yêu cầu xem xét tất cả 2^p các mô hình chứa s biến dự đoán. Do đó, chúng ta có thể diễn giải hồi quy Ridge và lasso như những lựa chọn thay thế khả thi về mặt tính toán cho việc lựa chọn tập con tốt nhất, thay thế dạng ngân sách khó xử lý trong (6.10) bằng các dạng Những bài toán đó dễ giải hơn nhiều. Tất nhiên, phương pháp dùng dây thòng lọng thì chính xác hơn nhiều. liên quan đến việc lựa chọn tập con tốt nhất, vì thuật toán lasso thực hiện việc lựa chọn đặc trưng với s đủ nhỏ trong (6.8), trong khi hồi quy Ridge thì không.

Thuộc tính lựa chọn biến của Lasso

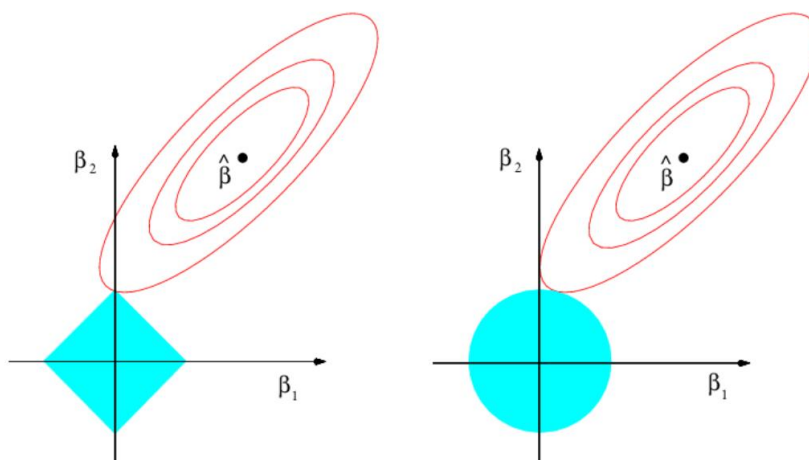
Tại sao phương pháp lasso, không giống như hồi quy ridge, lại cho ra các ước tính hệ số chính xác bằng không? Các công thức (6.8) và (6.9) có thể có thể được sử dụng để làm sáng tỏ vấn đề. Hình 6.7 minh họa tình huống.

Giải pháp bình phương tối thiểu được ký hiệu là $\hat{\beta}^*$, trong khi hình thoi và hình tròn màu xanh lam được ký hiệu là $\hat{\beta}^*$, còn hình thoi và hình tròn màu xanh lam được ký hiệu là $\hat{\beta}^*$.

biểu thị các ràng buộc hồi quy lasso và ridge trong (6.8) và (6.9) tương ứng. Nếu s đủ lớn, thì các vùng ràng buộc sẽ chứa $\hat{\beta}^*$, và do đó các ước tính hồi quy Ridge và Lasso sẽ giống như ước lượng bình phương tối thiểu. (Giá trị s lớn như vậy tương ứng với $\lambda = 0$ trong (6.5) và (6.7).) Tuy nhiên, trong Hình 6.7, các ước tính bình phương nhỏ nhất nằm ngoài hình thoi và hình tròn, do đó các ước tính bình phương nhỏ nhất là Không giống với các ước tính hồi quy lasso và ridge.

Mỗi hình elip có tâm tại $\hat{\beta}^*$ biểu thị một đường đồng mức: điều này có nghĩa là tất cả các điểm trên một hình elip cụ thể đều có cùng giá trị RSS. Như vậy,

đường viền



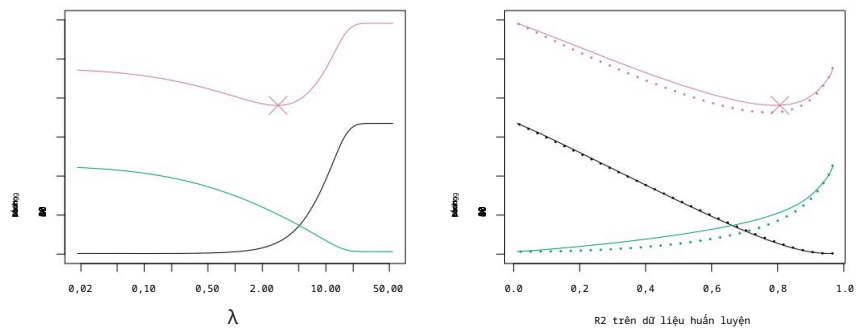
HÌNH 6.7. Đường đồng mức của hàm lỗi và hàm ràng buộc cho phương pháp lasso (trái) và hồi quy ridge (phải). Các vùng màu xanh lam liền nét là các vùng ràng buộc, $|\beta_1| + |\beta_2| \leq s$ và $\beta_1^2 + \beta_2^2 \leq s$, trong khi các hình elip màu đỏ là đường đồng mức của RSS.

Khi các hình elip mở rộng ra xa các ước tính hệ số bình phương nhỏ nhất, RSS tăng lên. Phương trình (6.8) và (6.9) chỉ ra rằng các ước tính hệ số hồi quy lasso và ridge được đưa ra bởi điểm đầu tiên mà một hình elip tiếp xúc với vùng ràng buộc. Vì hồi quy ridge có ràng buộc hình tròn không có điểm nhọn, nên giao điểm này thường sẽ không xảy ra trên một trục, và do đó các ước tính hệ số hồi quy ridge sẽ hoàn toàn khác không. Tuy nhiên, ràng buộc lasso có các góc ở mỗi trục, và do đó hình elip thường sẽ giao với vùng ràng buộc tại một trục. Khi điều này xảy ra, một trong các hệ số sẽ bằng không. Trong không gian nhiều chiều hơn, nhiều ước tính hệ số có thể bằng không đồng thời. Trong Hình 6.7, giao điểm xảy ra tại $\beta_1 = 0$, và do đó mô hình kết quả sẽ chỉ bao gồm β_2 .

Trong Hình 6.7, chúng ta đã xem xét trường hợp đơn giản khi $p = 2$. Khi $p = 3$, vùng ràng buộc cho hồi quy Ridge trở thành một hình cầu, và vùng ràng buộc cho thuật toán Lasso trở thành một đa diện. Khi $p > 3$, vùng ràng buộc cho hồi quy Ridge trở thành một siêu cầu, và vùng ràng buộc cho thuật toán Lasso trở thành một đa giác lồi. Tuy nhiên, những ý tưởng chính được mô tả trong Hình 6.7 vẫn đúng. Cụ thể, thuật toán Lasso dẫn đến việc chọn lọc đặc trưng khi $p > 2$ do các góc nhọn của đa diện hoặc đa giác lồi.

So sánh phương pháp Lasso và phương pháp hồi quy Ridge

Rõ ràng là phương pháp lasso có ưu thế vượt trội so với hồi quy Ridge, ở chỗ nó tạo ra các mô hình đơn giản và dễ hiểu hơn, chỉ bao gồm một tập hợp con các biến dự báo. Tuy nhiên, phương pháp nào mang lại độ chính xác dự đoán tốt hơn? Hình 6.8 hiển thị phương sai, độ lệch bình phương và MSE kiểm định của phương pháp lasso được áp dụng cho cùng dữ liệu mô phỏng như trong Hình 6.5. Rõ ràng phương pháp lasso dẫn đến hành vi tương tự về mặt định tính với hồi quy Ridge, ở chỗ khi λ tăng, phương sai giảm và độ lệch tăng. Ở phía bên phải



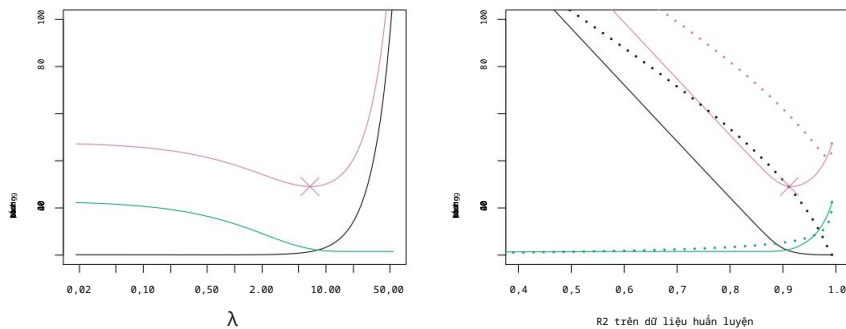
HÌNH 6.8. Bên trái: Đồ thị của độ lệch bình phương (màu đen), phương sai (màu xanh lá cây) và kiểm định MSE (màu tím) của thuật toán lasso trên tập dữ liệu mô phỏng. Bên phải: So sánh bình phương của... Độ lệch, phương sai và MSE kiểm định giữa lasso (đường liền nét) và ridge (đường chấm). Cả hai đều được biểu diễn trên đồ thị so với hệ số R2 của chúng trên dữ liệu huấn luyện, như một hình thức lập chỉ mục phổ biến. Dấu chéo trong cả hai biểu đồ cho biết mô hình lasso nào có MSE nhỏ nhất.

Trong hình 6.8, các đường chấm chấm biểu thị đường hồi quy Ridge phù hợp. Ở đây, chúng ta vẽ đồ thị cả hai giá trị so với hệ số R2 của chúng trên dữ liệu huấn luyện. Đây là một ví dụ khác. Đây là một cách hữu ích để lập chỉ mục các mô hình và có thể được sử dụng để so sánh các mô hình với nhau. Các loại chuẩn hóa khác nhau, như trường hợp này. Trong ví dụ này, Phương pháp hồi quy Lasso và Ridge cho kết quả sai lệch gần như giống hệt nhau. Tuy nhiên, Độ lệch chuẩn của hồi quy Ridge thấp hơn một chút so với độ lệch chuẩn của Lasso. Do đó, MSE tối thiểu của hồi quy Ridge nhỏ hơn một chút so với đó là cái thông lọng.

Tuy nhiên, dữ liệu trong Hình 6.8 được tạo ra theo cách sao cho tất cả 45 Các biến dự báo có liên quan đến biến phản hồi – nghĩa là, không có hệ số thực nào. $\beta_1, \dots, \beta_{45}$ bằng không. Thuật toán lasso ngầm giả định rằng một số trong số các các hệ số thực sự bằng không. Do đó, không có gì đáng ngạc nhiên khi sống núi Trong trường hợp này, hồi quy cho kết quả dự đoán sai số tốt hơn so với thuật toán Lasso. Hình 6.9 minh họa một tình huống tương tự, ngoại trừ việc bây giờ phản hồi là một Chức năng này chỉ phụ thuộc vào 2 trong số 45 biến dự đoán. Giờ đây, thuật toán Lasso có xu hướng hoạt động tốt hơn. Hồi quy Ridge xét về độ lệch, phương sai và MSE.

Hai ví dụ này minh họa rằng cả phương pháp hồi quy Ridge lẫn phương pháp Lasso đều không hiệu quả. sẽ hoàn toàn áp đảo bên kia. Nói chung, người ta có thể mong đợi chiếc thông lọng. để hoạt động tốt hơn trong mỗi trường hợp có số lượng yếu tố dự đoán tương đối nhỏ. có hệ số đáng kể, và các biến dự báo còn lại có hệ số có giá trị rất nhỏ hoặc bằng không. Phương pháp hồi quy Ridge sẽ cho kết quả tốt hơn. khi phản hồi là một hàm của nhiều biến dự báo, tất cả đều có hệ số là Kích thước gần bằng nhau. Tuy nhiên, số lượng biến dự báo có liên quan đến Phản hồi không bao giờ được biết trước đối với các tập dữ liệu thực tế. Một kỹ thuật như... Phương pháp kiểm định chéo có thể được sử dụng để xác định cách tiếp cận nào tốt hơn. trên một tập dữ liệu cụ thể.

Tương tự như hồi quy Ridge, khi các ước lượng bình phương nhỏ nhất có phương sai quá cao, giải pháp Lasso có thể giúp giảm phương sai. với chi phí là một sự gia tăng nhỏ về độ lệch, và do đó có thể tạo ra các dự đoán chính xác hơn. Không giống như hồi quy Ridge, Lasso thực hiện việc lựa chọn biến số, và do đó dẫn đến các mô hình dễ hiểu hơn.



HÌNH 6.9. Bên trái: Đồ thị của độ lệch bình phương (màu đen), phương sai (màu xanh lá cây) và kiểm định MSE (màu tím) cho thuật toán lasso. Dữ liệu mô phỏng tương tự như trong Hình 6.8. ngoại trừ việc giờ đây chỉ còn hai biến dự báo liên quan đến biến phản hồi. Bên phải: So sánh về độ lệch bình phương, phương sai và MSE kiểm định giữa lasso (đường liền nét) và ridge (đường chấm). Cả hai đều được biểu diễn trên đồ thị so với hệ số R^2 trên dữ liệu huấn luyện, như một hình thức phổ biến. lập chỉ mục. Các dấu chéo trong cả hai biểu đồ cho biết mô hình lasso mà MSE là nhỏ nhất.

Có những thuật toán rất hiệu quả để phù hợp với cả mô hình ridge và lasso; Trong cả hai trường hợp, toàn bộ đường dẫn hệ số có thể được tính toán với khoảng... Lượng công việc tương đương với một phép khớp bình phương tối thiểu đơn lẻ. Chúng ta sẽ tìm hiểu điều này. Sẽ có thêm phần thực hành trong phòng thí nghiệm ở cuối chương này.

Một trường hợp đặc biệt đơn giản cho hồi quy Ridge và Lasso

Để hiểu rõ hơn về hành vi của hồi quy Ridge, cần có sự hỗ trợ.

Và với thuật toán lasso, hãy xem xét một trường hợp đặc biệt đơn giản với $n = p$, và X là ma trận chéo với các phần tử trên đường chéo chính là 1 và các phần tử ngoài đường chéo chính là 0.

Để đơn giản hóa vấn đề hơn nữa, giả sử chúng ta cũng đang thực hiện hồi quy mà không có hệ số chặn. Với những giả định này, phương pháp bình phương tối thiểu thông thường...

Bài toán được đơn giản hóa thành việc tìm $\beta_1, \dots, \beta_p$ sao cho tối thiểu hóa

$$\sum_{j=1}^p (y_j - \beta_j)^2. \quad (6.11)$$

Trong trường hợp này, nghiệm bình phương tối thiểu được đưa ra bởi

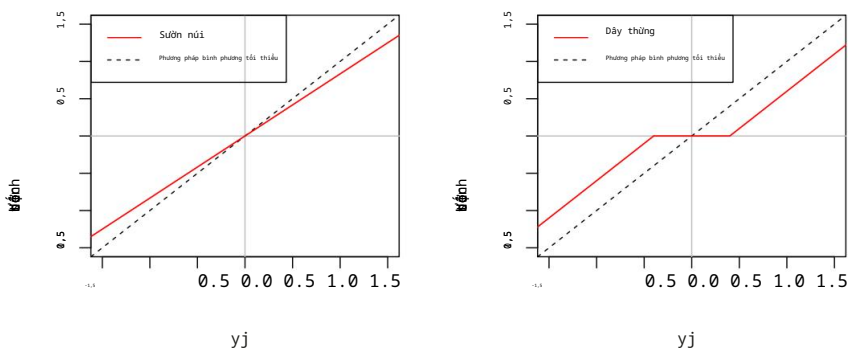
$$\beta^* = y.$$

Và trong bối cảnh này, hồi quy Ridge tương đương với việc tìm $\beta_1, \dots, \beta_p$ sao cho

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (6.12)$$

được tối thiểu hóa, và thuật toán lasso tương đương với việc tìm các hệ số sao cho

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \quad (6.13)$$



HÌNH 6.10. Ước tính hệ số hồi quy Ridge và Lasso cho một mô hình đơn giản.

Thiết lập với $n = p$ và X là ma trận đường chéo có các số 1 trên đường chéo. Bên trái: The Các ước tính hệ số hồi quy Ridge bị thu hẹp tỷ lệ thuận về 0. so với các ước tính bình phương nhỏ nhất. Bên phải: Các ước tính hệ số Lasso là Ngưỡng mềm hướng về số không.

được tối thiểu hóa. Có thể chứng minh rằng trong thiết lập này, các ước lượng hồi quy Ridge có dạng

$$\hat{\beta}_j^R = y_j / (1 + \lambda), \quad (6.14)$$

và các ước tính lasso có dạng

$$\hat{\beta}_j^L = \begin{cases} y_j - \lambda/2 & \text{nếu } y_j > \lambda/2; \\ y_j + \lambda/2 & \text{nếu } y_j < -\lambda/2; \\ 0 & \text{nếu } |y_j| \leq \lambda/2. \end{cases} \quad (6.15)$$

Hình 6.10 minh họa tình hình. Chúng ta có thể thấy rằng hồi quy Ridge và Công cụ lasso thực hiện hai loại co rút rất khác nhau. Trong hồi quy Ridge, Mỗi ước lượng hệ số bình phương tối thiểu đều bị thu nhỏ theo cùng một tỷ lệ. Ngược lại, phương pháp lasso thu nhỏ mỗi hệ số bình phương tối thiểu về phía 0 bằng cách... một lượng không đổi, $\lambda/2$; các hệ số bình phương nhỏ nhất nhỏ hơn $\lambda/2$ theo giá trị tuyệt đối bị thu nhỏ hoàn toàn về 0. Loại thu nhỏ được thực hiện bởi lasso trong thiết lập đơn giản này (6.15) được gọi là ngưỡng mềm. Việc một số hệ số lasso bị thu nhỏ hoàn toàn về 0 giải thích tại sao lasso thực hiện lựa chọn đặc trưng.

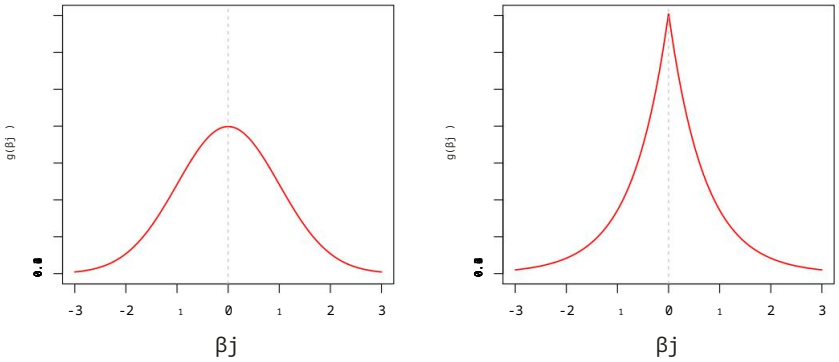
Trong trường hợp ma trận dữ liệu tổng quát hơn X , câu chuyện sẽ phức tạp hơn một chút. phức tạp hơn so với hình minh họa trong Hình 6.10, nhưng những ý chính vẫn giữ nguyên. Giá trị xấp xỉ: phương pháp hồi quy Ridge làm thu nhỏ mọi chiều. dữ liệu được chia theo cùng tỷ lệ, trong khi phương pháp lasso thì thu hẹp lại ít nhiều. Tất cả các hệ số đều tiến về 0 với một lượng tương tự, và các hệ số đủ nhỏ sẽ bị thu hẹp hoàn toàn về 0.

Giải thích Bayes về hồi quy Ridge và Lasso



Chúng tôi sẽ chứng minh rằng có thể xem xét hồi quy Ridge và Lasso thông qua Một góc nhìn Bayes. Quan điểm Bayes về hồi quy giả định rằng...

Vectơ hệ số β có một phân phối tiên nghiệm nào đó, chẳng hạn như $p(\beta)$, trong đó $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$. Xác suất của dữ liệu có thể được viết là $f(Y|X, \beta)$,



HÌNH 6.11. Bên trái: Hồi quy Ridge là giá trị cực đại hậu nghiệm cho β dưới phân phối tiên nghiệm Gaussian. Bên phải: Lasso là giá trị cực đại hậu nghiệm cho β dưới phân phối tiên nghiệm hàm mũ kép.

trong đó $X = (X_1, \dots, X_p)$. Nhân phân phối tiên nghiệm với xác suất sẽ cho ta (sai khác một hằng số tỷ lệ) phân phối hậu nghiệm, có dạng

phân phối
hậu nghiệm

$$p(\beta|X, Y) \propto f(Y|X, \beta)p(\beta) = f(Y|X, \beta)p(\beta),$$

trong đó tính tỷ lệ thuận nêu trên xuất phát từ định lý Bayes, và
Đẳng thức trên xuất phát từ giả thiết rằng X là cố định.

Chúng ta giả định mô hình tuyến tính thông thường,

$$Y = \beta_0 + X_1\beta_1 + \dots + X_p\beta_p + \epsilon,$$

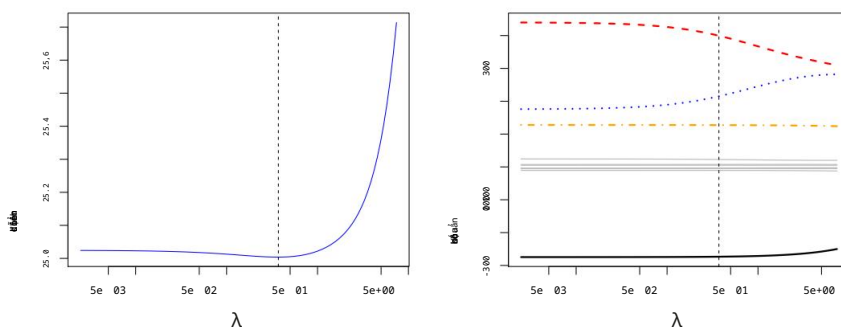
và giả sử rằng các sai số là độc lập và được lấy từ phân phối chuẩn. Hơn nữa, giả sử rằng
 $p(\beta) = g(\beta_j)$, với một số hàm mật độ p
hàm g . Hóa ra hồi quy Ridge và Lasso tuân theo quy luật tự nhiên.

từ hai trường hợp đặc biệt của g :

- Nếu g là phân phối Gaussian với giá trị trung bình bằng 0 và độ lệch chuẩn là một hàm của λ , từ đó suy ra rằng giá trị cực đại hậu nghiệm của β —tức là giá trị có khả năng nhất của β , dựa trên dữ liệu—được xác định bởi đường gờ.
giải pháp hồi quy. (Thực tế, giải pháp hồi quy Ridge cũng là...) trung bình hậu nghiệm.)
- Nếu g là phân phối mũ kép (Laplace) với giá trị trung bình bằng không. và tham số tỷ lệ là một hàm của λ , thì từ đó suy ra phân bố hậu nghiệm
Chế độ cho β là giải pháp lasso. (Tuy nhiên, giải pháp lasso không phải là giá trị trung bình hậu nghiệm, và trên thực tế, giá trị trung bình hậu nghiệm không mang lại kết quả nào. (Vectơ hệ số thừa.)

sau
cách thức

Các phân bố tiên nghiệm Gaussian và hàm mũ kép được hiển thị trong Hình 6.11.
Do đó, từ quan điểm Bayes, hồi quy Ridge và Lasso tuân theo
trực tiếp từ giả định mô hình tuyến tính thông thường với sai số chuẩn, cùng với
với phân phối tiên nghiệm đơn giản cho β . Lưu ý rằng phân phối tiên nghiệm Lasso có độ dốc lớn.
Đỉnh của đường cong nằm ở điểm 0, trong khi đường cong Gaussian phẳng hơn và dày hơn ở điểm 0. Do đó,
Lasso giả định trước rằng nhiều hệ số có giá trị (chính xác) bằng không, trong khi
Phương pháp Ridge giả định các hệ số được phân bố ngẫu nhiên xung quanh số 0.



HÌNH 6.12. Bên trái: Sai số kiểm định chéo phát sinh từ việc áp dụng thuật toán Ridge.

Hồi quy trên tập dữ liệu **Tín dụng** với các giá trị λ khác nhau. Bên phải: Hệ số

Các ước tính được biểu diễn dưới dạng hàm số của λ . Các đường chấm dọc biểu thị giá trị của λ .

Được lựa chọn bằng phương pháp kiểm định chéo.

6.2.3 Lựa chọn tham số điều chỉnh

Tương tự như các phương pháp lựa chọn tập con được xem xét trong Phần 6.1, cần phải...

một phương pháp để xác định mô hình nào trong số các mô hình đang được xem xét là tốt nhất.

Việc triển khai hồi quy Ridge và Lasso đòi hỏi một phương pháp để lựa chọn

một giá trị cho tham số điều chỉnh λ trong (6.5) và (6.7), hoặc tương đương với,

giá trị của ràng buộc s trong (6.9) và (6.8). Kiểm định chéo cung cấp một cách đơn giản để giải

quyết vấn đề này. Chúng ta chọn một lưới các giá trị λ và tính toán

lỗi kiểm định chéo cho mỗi giá trị của λ , như đã mô tả trong Chương 5. Chúng tôi

sau đó chọn giá trị tham số điều chỉnh sao cho lỗi kiểm định chéo là chính xác.

là nhỏ nhất. Cuối cùng, mô hình được hiệu chỉnh lại bằng cách sử dụng tất cả các quan sát có sẵn.

và giá trị đã chọn của tham số điều chỉnh.

Hình 6.12 hiển thị lựa chọn λ thu được từ việc thực hiện kiểm định chéo loại bỏ từng phần tử

trên các đường hồi quy Ridge phù hợp từ dữ liệu **Tín dụng**.

đã được thiết lập. Các đường thẳng đứng đứt đoạn biểu thị giá trị λ đã chọn. Trong trường hợp này

Giá trị này tương đối nhỏ, cho thấy sự phù hợp tối ưu chỉ liên quan đến một

Lượng co rút nhỏ so với nghiệm bình phương tối thiểu. Ngoài ra,

Sự sụt giảm không quá rõ rệt, do đó phạm vi giá trị khá rộng.

Điều đó sẽ gây ra lỗi tương tự. Trong trường hợp như vậy, chúng ta có thể đơn giản sử dụng

Giải pháp bình phương tối thiểu.

Hình 6.13 minh họa việc áp dụng phương pháp kiểm định chéo mười lần cho

Đường cong lasso khớp với dữ liệu mô phỏng thừa thớt từ Hình 6.9. Phía bên trái

Bảng bên phải của Hình 6.13 hiển thị lỗi kiểm định chéo, trong khi bảng bên phải hiển thị lỗi kiểm định chéo.

Bảng này hiển thị các ước tính hệ số. Các đường chấm dọc biểu thị...

Điểm mà tại đó lỗi kiểm định chéo là nhỏ nhất. Hai màu khác nhau

Các đường kẻ ở bảng bên phải của Hình 6.13 biểu thị hai biến dự đoán.

Các đường màu xám biểu thị các biến dự báo liên quan đến phản hồi, trong khi các đường màu xám

biểu thị các biến dự báo không liên quan; chúng thường được gọi là biến tín hiệu và biến nhiễu.

Không chỉ Lasso đã đưa ra các ước tính hệ số lớn hơn nhiều cho hai biến dự báo tín hiệu một

cách chính xác, mà sai số kiểm định chéo tối thiểu cũng tương ứng với một tập hợp các ước tính

hệ số mà chỉ có

các biến tín hiệu đều khác không. Do đó, cần sử dụng phương pháp kiểm định chéo cùng với...

Lasso đã xác định chính xác hai biến tín hiệu trong mô hình, thậm chí

Mặc dù đây là một bối cảnh đầy thách thức, với $p = 45$ biến và chỉ $n = 50$.

tín hiệu